FPT Education
FPT UNIVERSITY

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

Chart Question Answering System for Document Image Analysis

GSP26AI09	
Group Members	Mai Tuan Anh - Leader - SE182947
	Bui Xuan Hoang - Member - SE183251
	Le Quang That - Member - SE183256
Supervisor	Huynh Van Thong
Capstone Project code	SP26AI04

HCMC, April 2026

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background

In the modern information age, data is often referred to as a vital resource for decision making. However, raw data in its numerical form is often difficult for humans to perceive or analyze quickly. This challenge led to the widespread adoption of data visualization, specifically charts. Charts act as graphical representations of information, using visual symbols such as bars, lines, or slices to communicate complex relationships between data points.

Data visualization in the form of charts plays a pivotal role in data analysis, offering critical insights and aiding in informed decision making. Automatic chart understanding has witnessed significant advancements with the rise of large foundation models in recent years. Foundation models, such as large language models, have revolutionized various natural language processing tasks and are increasingly being applied to chart understanding tasks.

The context of this project is built upon three practical pillars. First is the necessity for efficient information retrieval. Users often need to extract specific data points from thousands of archived reports without manual searching. Second is the demand for automated report analysis. In corporate environments, an intelligent system that can automatically parse and summarize the key findings of a chart can save hours of human labor. Finally, there is a significant educational application in supporting writing skills. In countries like Vietnam, describing charts is a fundamental part of international English examinations such as IELTS. A tool that can analyze a chart and provide structured information or linguistic suggestions is highly valuable for students and educators.

Solving the problem of automated chart interpretation is of paramount importance as it democratizes data literacy. By allowing non technical users to query charts in natural language, we remove the technical barriers to data parsing. Furthermore, optimizing the system for Vietnamese specific datasets and common chart types such as Bar, Line, and Pie ensures that the solution is not only technologically advanced but also highly practical for local academic and professional contexts.

The decision to pursue this project is driven by the data trapped nature of visual documents. Despite the digital revolution, a vast amount of intelligence remains locked in static image formats, inaccessible to automated computation. The core problem we aim to address is the semantic disconnect between a chart pixels and its underlying logic. While traditional OCR can read text, it cannot comprehend how visual elements like a bar height correlate to numerical values. This project was chosen to bridge this gap by creating an integrated pipeline that transforms visual encodings into actionable knowledge, specifically focusing on a lightweight architecture that maintains high precision.

The Chart Question Answering task being developed in this work originates from the fundamental concepts of Visual Question Answering proposed by [1]. While VQA provides a general framework for reasoning over natural images, ChartQA specializes this idea by focusing on structured, data driven visual representations. By inheriting the core mechanism of integrating Computer Vision and Natural Language Processing, our ChartQA approach requires the model not only to recognize visual elements like bars, lines, or labels but also to perform logical and mathematical reasoning to derive precise answers from the charts. Understanding these needs helps clarify why this project aims to bridge the gap between static pixels and meaningful, actionable information.

1.2 Objective

The primary objective of this project is to build an automated Chart Question Answering system capable of understanding and extracting information from visual graphics. By bridging the gap between computer vision and natural language processing, the system allows users to query data using

natural language, ensuring that insights from complex charts are accessible, accurate, and efficient. To achieve the high level vision, the system is designed to meet the following technical and functional requirements:

1. **Data Processing and Structural Recognition:** The system must accurately identify core chart components, including titles, x and y axes, legends, and data points. It provides robust support for the three most prevalent formats: bar charts, line graphs, and pie charts. To ensure high system precision, the data is first transformed into a structured Markdown format before any further information processing or reasoning occurs.

2. **Advanced Language Understanding:** The system is designed to handle diverse query types related to the chart:

    - **Data Retrieval and Reasoning:** Understanding and answering queries ranging from direct information lookups to complex mathematical operations, such as calculating percentage changes or comparing data points.
    - **Narrative Summarization:** Synthesizing the chart visual data into a concise textual summary that captures the core message expressed by the chart.

3. **Performance and Model Efficiency:** The system focuses on achieving high accuracy through specialized components:

    - **Reasoning and Linguistic Precision:** Achieve state of the art performance in generating natural language responses, specifically optimized for Vietnamese language datasets. The system is evaluated using standard metrics to ensure high factual accuracy and linguistic fluency when interpreting chart related information.
    - **Efficient Pre classification:** Utilizing a compact classification model that achieves an optimal balance between low computational footprint and high predictive accuracy, ensuring precise identification of bar, line, and pie charts before data extraction.
    - **Data Reconstruction:** Achieve high accuracy in converting visual data into text by utilizing the state of the art *PaddleOCR-VL* model to extract precise Markdown tables from chart images.
    - **System Scale:** Optimizing the architecture as a Mini LLM with approximately 1.5B parameters to balance performance and computational efficiency.

4. **System Usability and Accessibility:** To ensure practical application, the project focuses on developing an intuitive and user friendly interface designed for everyone, including non technical users. By simplifying the interaction flow, the platform allows users to easily upload chart images and receive clear, natural language answers without requiring any specialized knowledge. This approach eliminates technical barriers, making complex visual data analysis accessible and straightforward for a broad range of users.

This project was chosen to bridge this specific gap through a Chart Question Answering system with a primary focus on the Vietnamese language. By enabling a machine to understand the visual language of charts and respond to natural language queries in Vietnamese, we can transform static images back into actionable knowledge for local users. Solving this issue is of great importance as it enhances the efficiency of data retrieval for domestic reports, supports automated business intelligence in the Vietnamese market, and improves accessibility for students or professionals who need to process large volumes of visual information rapidly in their native language.

1.3 Problem Statement

The core problem addressed in this project is the inability of traditional computational systems to directly read and reason over data stored in visual chart formats. While charts are efficient for human consumption, the data within them is computationally locked in pixel form, creating a barrier for automated analysis and interactive querying.

The project aims to develop a system that can autonomously transform these visual encodings into a machine readable structure to answer natural language questions. The challenge lies in executing this transformation with high precision while maintaining a lightweight architectural footprint of less than 2B parameters within a 12 week development cycle.

System Workflow and Constraints To solve this, the system must handle the following requirements:

1. **Input Validation:** Ensuring the system exclusively accepts supported digital chart formats and relevant queries, while filtering out non compliant inputs such as handwritten sketches or low quality scans to maintain processing integrity.

2. **Structural Decoding:** Bridging the gap between raw pixels and structured data in Markdown to ensure that the reasoning engine operates on factual, reconstructed values rather than visual approximations.

3. **Multi level Reasoning:** Generating natural language outputs that satisfy three distinct user needs: specific Data Retrieval, comparative Visual Reasoning, and Global Summarization of trends.

Ultimately, this project seeks to provide a seamless bridge between static chart images and actionable linguistic insights, ensuring that even non technical users can perform complex data analysis through simple, intuitive interactions.

1.4 Scope and Limitations

To establish clear expectations and ensure the integrity of the project objectives, the scope and limitations are defined as follows:

Project Scope The project encompasses the end to end development of a Chart Question Answering system, specifically focusing on:

1. **Target Chart Types:** The system is designed to classify and process three primary chart categories: bar charts, line charts, and pie charts.

2. **Input Format:** The scope is limited to high quality, digitally rendered images such as PNG, JPG, or JPEG typically found in electronic reports and publications.

3. **Information Extraction:** A structured pipeline that converts visual data into Markdown tables to maintain numerical precision before any reasoning occurs.

4. **Query Categories:** Providing accurate responses for data retrieval of exact values, visual reasoning involving comparisons or extremes, and global summarization of trends.

5. **Accessibility:** Developing an intuitive interface that allows non technical users to interact with the system without requiring specialized technical skills or domain knowledge.

Limitations To maintain focus on the core objectives within the 12 week timeframe, the following are excluded from the current project scope:

1. **Excluded Chart Formats:** The system will not cover complex or specialized charts such as 3D charts, scatter plots, radar charts, or geographic maps.
2. **Input Constraints:** Hand drawn sketches, low resolution physical scans, and handwritten annotations are outside the processing capabilities of the current extraction model.
3. **Multiple Charts:** The current version is optimized for single chart analysis per query and does not support cross referencing between multiple different chart images simultaneously.
4. **Language Support:** While the reasoning engine is optimized for Vietnamese specific datasets, support for other low resource languages is not guaranteed in this phase.

2 Project Management Plan

2.1 Team Structure and Roles

The project is managed by a team of three members with responsibilities divided among core areas: Engineering, Data, and Evaluation. The specific roles and tasks assigned to each member are as follows:

1. **Mai Tuan Anh**: Responsible for research on QA methodology, merging LoRA adapters, and leading the testing and evaluation of model performance.
2. **Bui Xuan Hoang**: Responsible for research on QA methodology, building the system pipeline, constructing the ChartQA dataset, and fine tuning the QA model.
3. **Le Quang That**: Responsible for collecting crawl data, building training datasets, labeling images, and training the chart classification model.

The team utilizes the following communication and management methods to ensure efficiency:

1. **Zalo**: Used for quick daily communication and immediate project updates.
2. **Google Meet**: Facilitates in depth discussions and virtual meetings.
3. **GitHub**: Used for managing source code versions and collaborative development.
4. **Google Drive**: Serves as a repository for managing documentation and shared resources.

2.2 Communication Plan

The team follows a structured communication plan to maintain alignment throughout the project lifecycle:

1. **Internal Communication**: The team maintains continuous daily interaction to update model training progress and address any technical errors immediately.
2. **Advisory Communication**: The team holds regular meetings with the instructor every Thursday to report on achieved milestones and receive expert advice for the next stages.
3. **Quality Management**: Reports and experimental results are cross checked by all team members to ensure accuracy before being presented to the supervising teacher.

3 Literature Review (Background and Relate work)

3.1 Background

3.1.1 Chart

Definition A chart (sometimes known as a graph) is a graphical representation for data visualization, in which "the data is represented by symbols, such as bars in a bar chart, lines in a line chart, or slices in a pie chart". A chart can represent tabular numeric data, functions or some kinds of quality structure and provides different info.[2]

Taxonomy and Structural Properties This research focuses on the three most prevalent chart types, each characterized by its unique geometric and logical structure:

1. **Bar Charts:** Use the length or height of rectangular bars to compare discrete quantities across different categories.

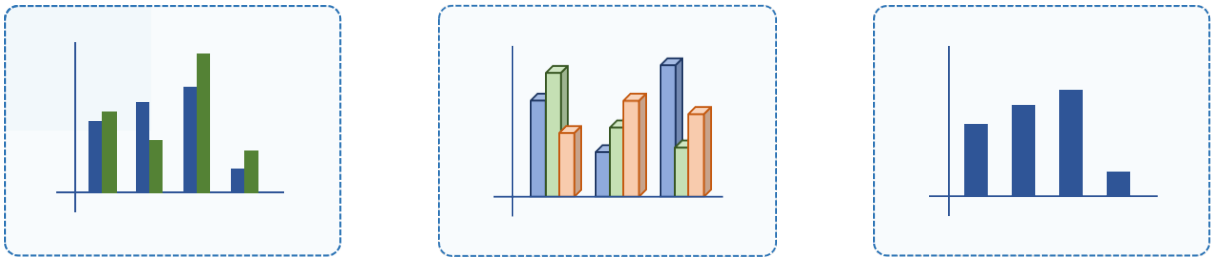


Figure 1: Examples of Bar Charts representing discrete data comparisons [3].

2. **Line Graphs:** Connect individual data points with line segments to highlight continuous trends, patterns, and fluctuations over time.

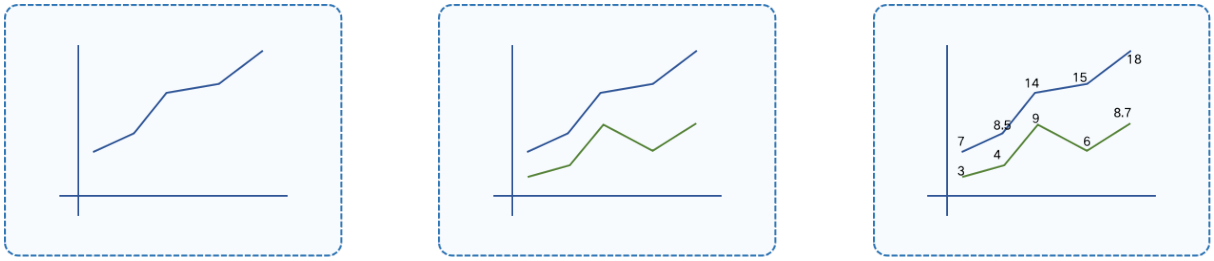


Figure 2: Examples of Line Graphs illustrating temporal trends [3].

3. **Pie Charts:** Partition a circle into proportional sectors to represent the "part to whole" relationship of a dataset.

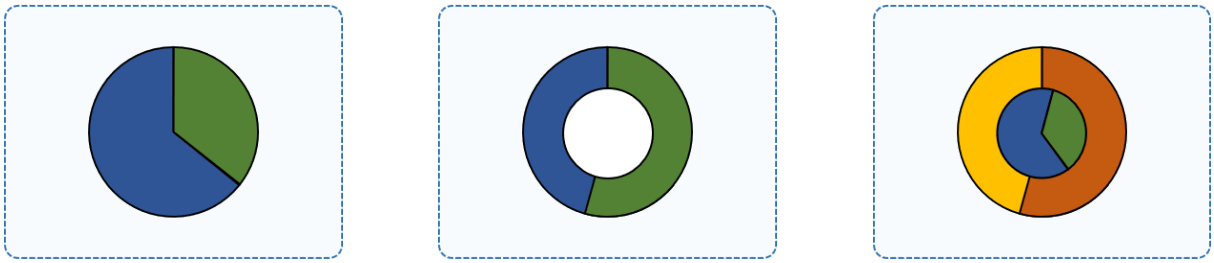


Figure 3: Examples of Pie Charts displaying proportional distributions [3].

Why Charts are Essential The primary motivation for incorporating charts into reports is to enhance *data digestibility*. Visual representations allow humans to recognize patterns, outliers, and correlations much faster than scanning raw tables. When users interact with a chart, they typically attempt to extract three levels of intelligence:

1. **Specific Values:** Retrieving exact data points associated with labels.
2. **Relational Insights:** Identifying the maximum, minimum, or rank of different categories.
3. **Strategic Trends:** Comprehending the overall trajectory or distribution of the data narrative.

3.1.2 Automatic Chart Understanding

Automatic Chart Understanding is the computer systems that understand human question and extract meaningful and exact information from graphical representations to answer it.

By automating this extraction, we transform static images into dynamic data sources, enabling the system to perform precise reasoning and complex data driven analysis that would otherwise require manual human intervention. This automated approach provides the foundational accuracy needed for subsequent tasks like natural language querying and trend synthesis.

3.1.3 Multimodal Language Model

A multimodal language model is a neural network that jointly processes inputs from multiple modalities, typically vision and language, within a single unified architecture. Unlike pipeline systems that rely on separate, independently trained components for image understanding and text generation, a multimodal model learns to align visual and linguistic representations during training, enabling it to reason over image content and produce natural language responses in an end to end manner.

The general architecture of such models consists of three core components. A visual encoder extracts structured feature representations from the input image. A projection module, commonly implemented as a multi layer perceptron, maps these visual features into the same embedding space as the language model so that visual information can be interpreted as tokens by the language decoder. Finally, a large language model receives the combined visual and textual token sequence and generates the output response. This architecture has become the dominant paradigm for visual question answering, document understanding, and chart analysis tasks, as it allows the model to ground its language generation directly in visual evidence rather than relying on intermediate representations that may lose detail.

For the question answering component of this work, a multimodal language model is adopted as the core reasoning module. The model must simultaneously interpret the visual structure of a chart and produce a coherent, accurate answer in Vietnamese, a requirement that a text only language model cannot satisfy. The model selected for this purpose is Vintern-1B-v2, described below.

Vintern-1B-v2 **Vintern-1B-v2** [4] is a 1 billion parameter multimodal language model developed by 5CD-AI, built on the InternVL 2.0 architecture. It accepts both images and text as input and generates natural language responses, making it well suited for chart understanding and Vietnamese visual question answering tasks.

As illustrated in Figure 4, the model consists of three fundamental components:

1. **Vision Encoder (InternViT-300M-448px):** A distilled vision foundation model is employed to extract deep visual features from input chart images. It is pre trained to handle a standard resolution of 448×448 pixels per tile.

2. **MLP Projector:** A two layer Multi Layer Perceptron acts as a bridge between modalities. It maps the visual feature representations into the same embedding space as the language model, ensuring the language model can interpret visual data as visual tokens.

3. **Large Language Model (Qwen2-0.5B-Instruct):** The reasoning core is powered by the Qwen2-0.5B model. This component receives the aligned representations and generates precise Vietnamese textual responses based on the visual context.

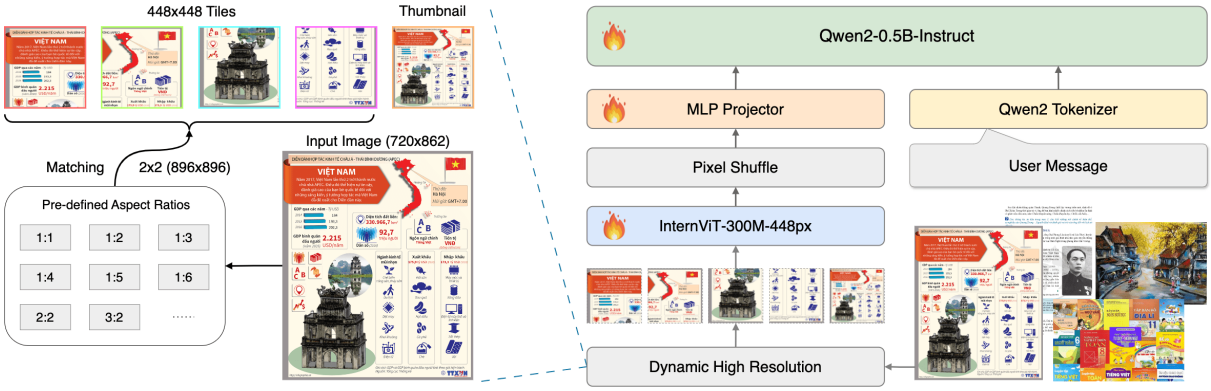


Figure 4: The overall architecture of Vintern-1B, featuring the integration of InternViT, an MLP Projector, and the Qwen2 LLM core.

Dynamic High Resolution Processing

To handle the intricate details of academic charts such as small legends and axis labels, the model implements a sophisticated image processing pipeline:

1. **Dynamic Tiling:** Following the InternVL 1.5 protocol, input images are segmented into 448×448 pixel tiles. This allows the model to scale its resolution according to the complexity of the chart.

2. **Pixel Shuffle Technique:** To maintain computational efficiency while scaling to high resolutions, a pixel shuffle operation is applied. This reduces the total token count by compressing the spatial dimensions of the visual features.

3. **Token Representation:** Through this pipeline, each 448×448 image segment is encoded into exactly 256 visual tokens, which are then interleaved with textual instructions for processing by the language decoder.

3.1.4 Classification

Chart classification is the task of assigning a chart image to one of a predefined set of chart type categories, such as bar, line, pie, or scatter. It serves as the entry point of the analysis pipeline, determining which downstream processing strategy should be applied to a given input. A misclassification at this stage routes the image to an incorrect handler, causing cascading errors throughout the system. The task is therefore both foundational and time sensitive, as it must be resolved reliably before any further analysis can proceed.

From a visual recognition perspective, chart classification is well suited to convolutional neural network based approaches, as the distinguishing features between chart types, such as the circular geometry of pie charts, the vertical or horizontal bars of bar charts, and the continuous curves of line charts, are perceptually salient and spatially structured. These characteristics make it possible to achieve strong classification performance with relatively compact architectures, without requiring the deep feature hierarchies that more abstract recognition tasks demand.

Residual Network (ResNet) ResNet [5] is a family of deep convolutional neural networks that introduces *shortcut connections* to bypass one or more layers, allowing the network to learn residual mappings rather than direct feature transformations. This design effectively mitigates the vanishing gradient problem that hinders the training of very deep networks, enabling the construction of significantly deeper and more accurate architectures without a degradation in performance.

Among its variants, **ResNet-18** is the most lightweight, comprising 18 weight layers and approximately 11 million parameters. Its compact size makes it computationally efficient and well suited for training on domain specific datasets of moderate scale. For chart type classification, where the visual differences between categories are relatively distinctive and do not require highly abstract feature hierarchies, ResNet-18 provides sufficient representational capacity while avoiding the risk of overfitting that deeper variants may introduce on smaller datasets [5].

3.1.5 Chart Parsing

Chart parsing refers to the process of extracting structured, machine readable information from a chart image, recovering elements such as axis labels, legends, data series, and numerical values in a form that downstream reasoning components can directly consume. Unlike general image captioning or visual question answering, chart parsing demands precise localisation and recognition of dense textual and graphical elements, often at small scales and with significant layout variation across chart types. This makes it a specialised and technically demanding subtask that general purpose vision encoders are not optimised to handle reliably.

Accurate chart parsing is a critical prerequisite for the question answering pipeline developed in this work. Errors introduced at the parsing stage, such as misread axis values or dropped data series, propagate directly into the reasoning process and cannot be recovered by the language model downstream. The choice of parsing component therefore has an outsized impact on overall system performance.

PaddleOCR-VL Chart Question Answering requires a system to not only understand the visual layout of a chart but also reason precisely over the numerical data it encodes. A critical bottleneck in this process is the extraction of accurate, structured data from chart images, a task that general purpose vision encoders are not optimised for. PaddleOCR-VL [6] addresses this bottleneck directly as a state of the art document parsing model developed by the PaddlePaddle team at Baidu.

Its architecture combines a NaViT style [7] dynamic high resolution visual encoder that processes images at arbitrary resolutions and aspect ratios without resizing, with the lightweight ERNIE-4.5-0.3B [8] language model. This design enables accurate recognition of complex document elements including charts, tables, formulas, and dense text at minimal computational cost. Of particular relevance to Chart QA, PaddleOCR-VL is trained to parse chart content into structured Markdown and JSON output, explicitly recovering data series, axis labels, legends, and numerical values in a format that language models can directly consume for question answering.

PaddleOCR-VL has been evaluated on widely used public benchmarks including OmniDocBench v1.0 and v1.5, where it achieves state of the art performance in both page level document parsing and element level recognition, significantly outperforming existing pipeline based solutions and exhibiting strong competitiveness against top tier vision language models [6]. Its chart recognition capability in particular ranks among the best available tools across diverse chart types. Given this level of benchmark validated performance in the exact task our pipeline depends on, we adopt PaddleOCR-VL directly as the chart parsing component without comparing against alternative tools, as no existing open solution has demonstrated superior chart parsing quality at comparable efficiency.

3.2 Related Tasks

Natural Image Understanding Natural image understanding is dedicated to interpreting photographic data that captures real world environments, including landscapes, human activities, and objects. The core objective is to develop computational models capable of replicating human visual perception. Prominent research areas in this field include image captioning [9, 10] and Visual Question Answering (VQA) [11, 12].

The fundamental divergence between natural image and chart understanding lies in the character of the visual information. While natural image tasks deal with unstructured data influenced by environmental factors like lighting and occlusions, chart understanding focuses on structured, symbolic representations. Unlike general VQA, which often prioritizes object recognition, chart interpretation centers on precise data extraction, identifying graphical primitives such as bars or lines, and decoding the quantitative narrative embedded within the visualization. This necessitates not only visual recognition but also high level analytical and mathematical reasoning.

Table Understanding This field focuses on extracting and interpreting information from data arranged in tabular formats [13, 14, 15, 16]. The primary challenges involve recognizing layout structures, establishing row column relationships, and understanding the semantic context of cell values. While both tables and charts represent structured data, tables offer a direct, text based view of information.

In contrast, chart understanding introduces a layer of visual encoding using attributes like color, orientation, and scale to represent data. These encodings allow for immediate perception of trends and distributions that are less obvious in raw tables, but they necessitate advanced visual reasoning. Although one might attempt to simplify chart understanding by converting graphics back into tables, this approach is often hindered by the lack of accompanying raw data in publications and the high error rates of automated extraction tools, reinforcing the need for specialized chart to text or ChartQA models.

Document Understanding Visual Document Understanding (VDU) involves the holistic analysis of visually presented documents, ranging from scanned receipts and digital PDFs to academic slides [17, 18, 19]. VDU models must simultaneously decode textual content and spatial layouts, capturing the interplay between headings, figures, and footnotes to grasp the document's overall message.

The distinction between VDU and chart understanding is primarily one of scope. While VDU addresses a wide array of document elements and their arrangements, chart understanding is a specialized domain focused strictly on graphical data representations. Furthermore, charts are increasingly used as standalone objects in digital media and web environments, independent of traditional document structures. This ubiquity demands robust models that can interpret visual data autonomously, without relying on broader document context.

4 Methodology

4.1 Overview

The proposed Chart Question Answering (CQA) system is designed as a modular, three-stage pipeline to achieve precise numerical data retrieval and reasoning over chart images. Our approach prioritizes a deterministic data-extraction phase to preserve high accuracy before engaging the large language model for language synthesis. The workflow is entirely sequential, converting raw pixels into a structured knowledge base, which then serves as the ground-truth input for user querying.

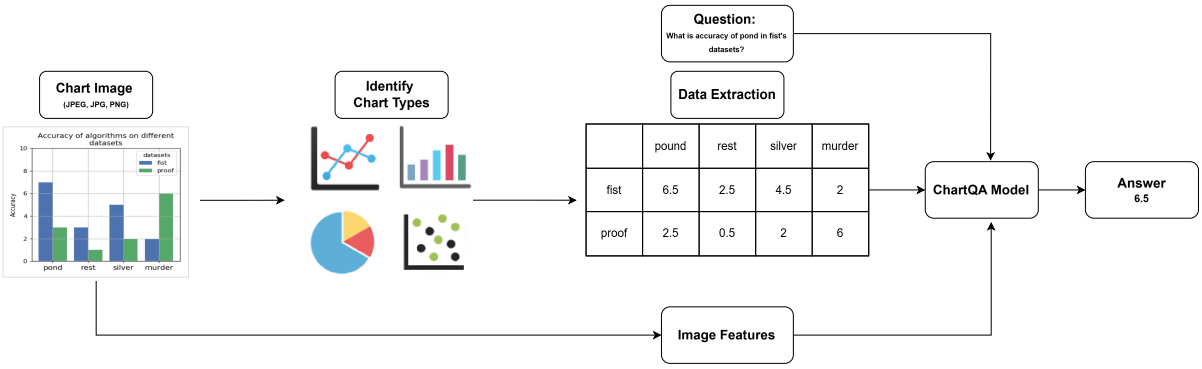


Figure 5: The proposed architectural pipeline for the Chart Question Answering system, illustrating the flow from image input to natural language response.

Stage 1: Input Validation and Chart Type Identification The pipeline begins with the ingestion of a digital chart image. To maintain processing integrity, an automatic validity check is performed to ensure the input is a supported digitally rendered chart and not noise. Following validation, a lightweight classification model is deployed to accurately distinguish between the three target chart formats: bar, line, and pie charts. This pre-classification is a critical optimization step, as it enables the system to route the image to a subsequent extraction module specifically tailored for that chart's unique visual structure.

Stage 2: High-Precision Visual-to-Structured Data Conversion To ensure the final reasoning engine operates on precise numerical values, this stage handles the semantic decoding of visual elements. Based on the chart type identified, we utilize the state-of-the-art *PaddleOCR-VL* model. Prior to any further information processing, this model converts the image's key visual features (bars, lines, slices, labels, and axes) directly into a structured Markdown table. By "unlocking" the data from its pixel format, we establish a robust factual foundation, mitigating the risk of data hallucination.

Stage 3: Vietnamese Language Understanding and Multi-level Reasoning The final stage bridges the gap between structured data and human intuition. The extracted Markdown table is fed into a fine-tuned *Mini-LLM* (less than 2B parameters), which is optimized for Vietnamese-language understanding. This compact reasoning engine analyzes the user's natural language query and synthesizes an accurate response across three categories: direct data retrieval, comparative visual reasoning, and global summarization.

4.2 Implementation Details

4.2.1 Setup

The computational framework for this project was developed using a hybrid environment, combining local hardware for specialized training and cloud-based resources for large-scale evaluation. The hardware specifications are detailed as follows:

1. **NVIDIA GeForce RTX 3060 Laptop GPU:** This local hardware was dedicated to training the lightweight classification model. Featuring 6GB of GDDR6 memory and based on the Ampere architecture, this GPU provided sufficient power for the rapid iteration and optimization of the pre-classification stage, ensuring the model remains compact yet accurate.
2. **NVIDIA Tesla P100 (Kaggle):** Primarily utilized for the intensive training and fine-tuning of the PaddleOCR-VL and the Mini-LLM modules. With 16GB of high-bandwidth HBM2 memory, the P100 efficiently handled the large-scale tensor computations required to align visual data with linguistic structures.
3. **NVIDIA Tesla T4 (Kaggle):** This GPU was employed for the critical **evaluation phase** of the LLM. Leveraging the Turing architecture and 16GB of GDDR6 memory, the T4's specialized Tensor Cores and efficiency in mixed-precision (FP16) computations allowed for a high-throughput benchmarking of the model's reasoning accuracy and response generation.

This diverse hardware setup ensured that each component of the pipeline was developed and validated on an optimized platform, maintaining high precision across all 12 weeks of the project lifecycle.

4.3 Data

4.3.1 Overview

Our pipeline operates through three specialized models. Due to the superior performance of **PaddleOCR-VL** [20] in multilingual document parsing and chart-to-table conversion, it is integrated as a pre-trained, "off-the-shelf" engine. Since this model already delivers top-tier accuracy in data extraction, further fine-tuning was unnecessary. Consequently, our data collection and preparation efforts were strategically focused on the remaining two stages, requiring two distinct datasets:

Classification Dataset (For Image Categorization) To train the initial classification model, we curated a dataset by crawling academic figures from **arXiv**. This dataset is essential for the system to correctly route inputs to the appropriate extraction logic.

Reasoning Dataset (For Vintern-1B-v2 Fine-tuning) The second dataset was developed to fine-tune *Vintern-1B-v2*, an improved version based on the *Vintern-1B* architecture. As introduced by [21], Vintern-1B is an efficient multimodal large language model specifically optimized for the Vietnamese language. To adapt this model for complex Chart QA tasks, we utilized a hybrid dataset strategy:

1. **Vietnamese Chart QA Benchmark:** An open-source collection providing foundational chart reasoning patterns and linguistic structures in Vietnamese.
2. **Self-Built Synthetic Data:** A proprietary set of chart-table-query triplets. We specifically designed this data to cover complex reasoning paths, such as identifying long-term trends and performing comparative analysis. This ensures the model can synthesize sophisticated, natural language answers from the structured data provided by the extraction engine.

4.3.2 Reasoning Dataset (For Vintern-1B-v2 Fine-tuning)

The development of the reasoning engine requires a dataset that bridges the gap between structured table data and Vietnamese natural language. To achieve this, we utilize a hybrid data strategy, primarily anchored by the *Viet-Chart-VQA* benchmark.

Viet-Chart-VQA Dataset The primary foundation for the reasoning stage is the **Viet-Chart-VQA** dataset, an open-source benchmark specifically curated for Vietnamese chart-based visual question answering [22]. This dataset provides a diverse collection of charts and natural language pairs, which is essential for training the model to understand both visual structures and Vietnamese linguistic nuances. Notably, the dataset's comprehensive coverage includes the three core chart types targeted in this project: **bar charts** (both horizontal and vertical), **line charts**, and **pie charts**.

The dataset is organized into a multi-modal format with the following key components:

1. **Image and Metadata:** The dataset includes high-quality chart images. Each image is accompanied by its dimensions and precise bounding box (*plot-bb*) information for visual elements.
2. **Natural Language Descriptions:** Every chart is paired with a detailed paragraph in Vietnamese (*description*). These descriptions summarize the chart's content, identifying the axes, units, and prominent trends, which serves as a rich semantic context for the model.
3. **Conversational Structure:** The core of the dataset consists of *conversations* in a multi-turn format. These include:
    - **User Queries:** Questions ranging from simple data retrieval to complex comparative reasoning.
    - **Assistant Responses:** Ground-truth answers provided in natural Vietnamese, incorporating both numerical values and descriptive context.
4. **Structured Ground Truth:** A unique feature of this dataset is the inclusion of structured data-series or Markdown-like tables. This allows the model to learn the direct mapping between visual coordinates and their corresponding logical values.

Self-Built Vietnamese Chart Dataset While the *Viet-Chart-VQA* benchmark provides a valuable cross-lingual foundation, a significant limitation is that its chart images primarily contain English labels and annotations, while only the question-answering pairs are in Vietnamese. To ensure the reasoning engine can handle authentic, localized documents, we curated a supplementary dataset consisting of **native Vietnamese chart images** paired with Vietnamese queries. This dataset is crucial for bridging the visual-linguistic gap.

The development of this dataset focused on deep analytical depth. Each chart image in this collection is paired with **five distinct Question-Answer (QA) pairs**, designed to cover three essential levels of information processing:

1. **Data Retrieval:** Questions focusing on extracting specific, atomic data points directly from the chart. This ensures the model's basic lookup accuracy and its ability to identify precise numerical values.
2. **Global Overview:** Queries that require the model to synthesize a general narrative or identify the overall purpose of the chart. This trains the model to understand the context and the primary subject matter of the visual input.

3. **Data Reasoning:** Complex questions requiring the model to perform mathematical operations or logical comparisons based on the chart's data. This forces the model to reason over the structured table generated by the extraction engine to derive non-trivial insights.

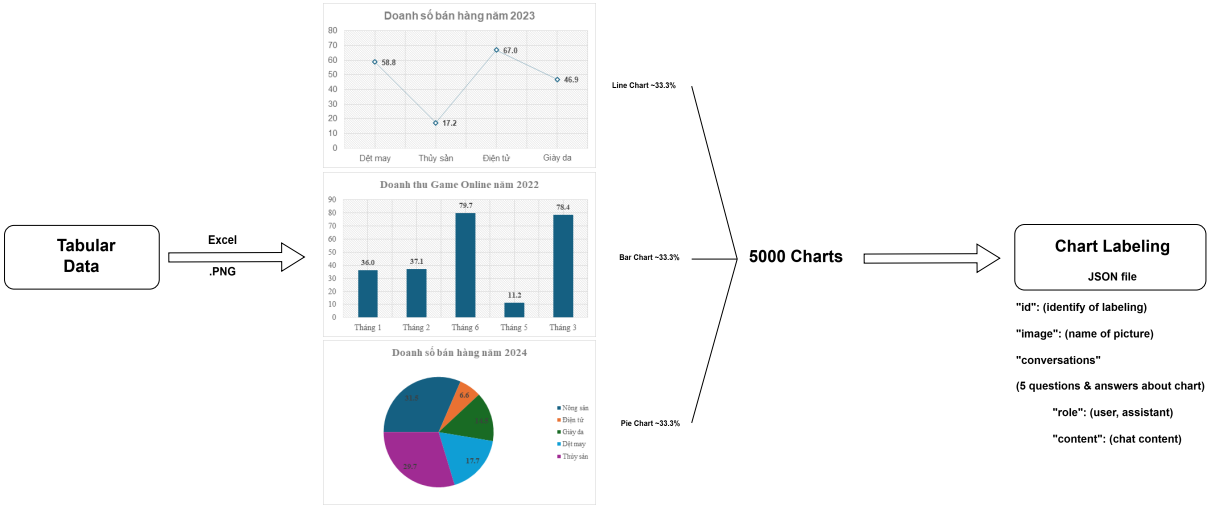


Figure 6: Overview of the self-built dataset construction process.

By introducing this self-built dataset, the *Vintern-1B-v2* model is not merely translating English data into Vietnamese but is genuinely learning to interpret Vietnamese visual information. The model is trained to recognize and process native terminology directly from the image pixels to provide accurate responses. This 5-pair-per-image structure ensures that for every visual input, the model provides a multi-dimensional analysis, significantly enhancing its utility for professional and educational applications within the Vietnamese digital ecosystem.

Our self-built data introduces several layers of complexity. Each assistant response is synthesized as a complete, grammatically correct Vietnamese narrative, rather than a single numerical value. This comprehensive fine-tuning ensures that the final model can handle the entire logical chain—from categorization to sophisticated, human reasoning effectively bridging the gap between static pixels and actionable information.

Data Preparation for Training The training data is compiled from two sources. The first is the **Viet-Chart-VQA Dataset**, a publicly available benchmark for Vietnamese chart question answering, in which each sample pairs a chart image with one question and one corresponding answer. The second is the **Self-Built Vietnamese Chart Dataset**, a manually curated collection assembled to supplement the public data with additional Vietnamese chart content, where each sample consists of a chart image accompanied by one or more question–answer pairs.

Before the two sources can be combined, a cleaning step is applied to the Self-Built Vietnamese Chart Dataset. Because this dataset was constructed by hand from heterogeneous sources, the way questions and answers are labeled varies across samples. A standardization step is therefore applied to bring every sample into the same uniform format, ensuring that each turn is unambiguously identified as either a question or an answer.

Additionally, some samples in the self-built data contain several question–answer pairs associated with a single chart image, whereas the Viet-Chart-VQA Dataset uses exactly one pair per sample. To ensure structural consistency, every multi-question sample is broken down into separate independent samples, one per question–answer pair, each retaining the original chart image.

Once both datasets share a consistent structure, they are merged. Because the Viet-Chart-VQA Dataset is substantially larger, only 30,000 randomly selected samples from its training split are included, preventing it from dominating the training signal. From the Self-Built Vietnamese Chart Dataset, 200 samples are reserved for evaluation and the remainder are used for training. The final training set thus combines 30,000 Viet-Chart-VQA samples with the self-built training samples, while the final test set combines the original Viet-Chart-VQA test split with the 200 reserved self-built samples.

Finally, all chart images are resized to 448 × 448 pixels to match the input resolution expected by the vision encoder, and every sample is exported as a pairing of an image file and a structured annotation that explicitly identifies the question and the expected answer, as required by the fine-tuning framework.

Final test split The test split used for evaluation was formed by combining 500 held out samples from the **Viet-Chart-VQA** dataset with 200 samples from the **self-built Vietnamese dataset**, yielding a total of **700 chart samples**.

4.3.3 Classification Dataset

The development of the classification module began with the construction of a high-quality training dataset derived from raw academic papers sourced from the **arXiv** [23] open-access repository. This process ensures that the model can accurately categorize input images into specific chart types before they are passed to the extraction engine.

Figure 7: The automated data collection and processing pipeline.

Data Collection Pipeline The training data was curated through a systematic pipeline consisting of the following stages:

1. **PDF Mining:** Approximately 4,000 academic papers were collected from the arXiv categories *cs.CV*, *cs.LG*, and *cs.AI*. These PDFs were rendered at 150 DPI using the PyMuPDF [24] library, a high-performance Python library for data extraction, analysis, conversion, and manipulation of PDF documents, resulting in approximately 150,000 page images.

2. **Chart Detection:** A YOLOv8m [25] model with a confidence threshold of 0.5 was employed to detect potential chart regions within the page images, yielding approximately 70,000 candidate crops.

3. **Quality Filtering and Classification:** To ensure dataset integrity, our team manually reviewed and classified each crop into one of eight distinct chart types. After removing low-confidence detections and duplicates, we successfully retained 32,364 verified charts for further processing.

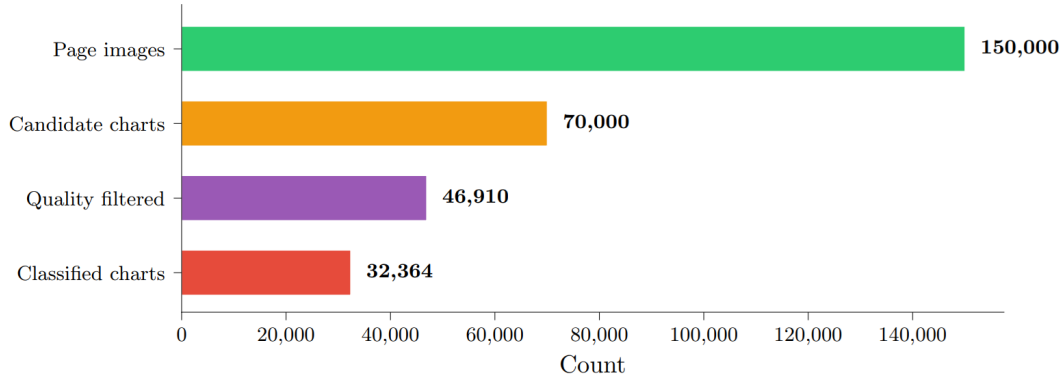


Figure 8: Data pipeline funnel illustrating the volume of data at each processing stage, from raw arXiv PDFs to final classified charts.

Chart Type Distribution The final classified dataset comprises 32,364 charts. The distribution reflects the prevalence of various data visualization styles in academic literature, with a strong emphasis on line, scatter, and bar charts. The detailed statistics are summarized in Table 1.

Table 1: Distribution of chart types in the final classified dataset.

Chart Type	Count	Share (%)
Line	12,930	39.9%
Scatter	6,278	19.4%
Bar	5,745	17.7%
Heatmap	4,073	12.6%
Histogram	1,006	3.1%
Box	880	2.7%
Pie	835	2.6%
Area	617	1.9%
Total	**32,364**	**100.0%**

4.4 Approach

Our approach focuses on transforming a pre-trained Vision Language Model (VLM) into a specialized engine for Vietnamese chart interpretation. This involves leveraging state of art architectures and efficient fine-tuning techniques to handle multi-modal inputs.

4.4.1 Classification

Although the primary scope of this work focuses on three chart types (*Pie*, *Bar*, and *Line*), a reliable classification module must be capable of recognising a broader range of chart forms encountered in real-world documents. In practice, input images may contain chart types outside the target scope, and misidentifying them as one of the three target classes would propagate errors throughout the subsequent analysis pipeline. To address this, the classifier is trained to distinguish eight commonly occurring chart types, enabling the model to correctly reject out-of-scope charts and route only the relevant ones for further processing. The eight categories considered are: *area*, *bar*, *box*, *heatmap*, *histogram*, *line*, *pie*, and *scatter*.

To this end, we employ ResNet-18 [5], a compact yet effective residual convolutional neural network that has demonstrated strong performance across a wide range of image recognition bench-

marks [27]. Rather than training from random initialisation, the backbone is initialised with weights pre-trained on the ImageNet dataset [28], allowing the network to leverage rich low and mid-level visual representations and converge more reliably on the relatively limited chart dataset [29].

Model Architecture ResNet-18 is structured as a feature extraction backbone followed by a classification head. In its original form, the classification head is a single fully connected layer that projects the 512-dimensional feature vector produced by the backbone to 1,000 ImageNet categories. In this work, that head is replaced by a two-layer module: a dropout layer [30], which randomly zeroes activations during training with probability $p = 0.3$ to reduce overfitting, followed by a linear layer that projects the 512-dimensional feature vector to the eight target classes. The backbone weights remain initialised from the ImageNet pre-trained checkpoint throughout this modification.

Pre-processing and Data Augmentation All input images are resized to 256×256 pixels prior to further processing. During training, a sequence of stochastic transformations is applied to each image to improve the model's ability to generalise to unseen data [31]. Specifically, a random 224×224 crop is extracted from the resized image to introduce spatial invariance. The cropped image is then subject to a random horizontal flip with probability $p = 0.2$, a random rotation within $\pm 15°$ to promote robustness for pie charts without distorting bar and line layouts, and a colour jitter operation that perturbs brightness and contrast by up to ± 0.2 and saturation by up to ± 0.1. Finally, a random affine transformation is applied with a translation of up to 5% of the image dimensions and a scale factor drawn uniformly from $[0.95, 1.05]$.

At inference time, no stochastic transformations are applied: images are resized and centre-cropped to 224×224 to ensure deterministic and reproducible predictions. All images, in both training and inference, are normalised channel-wise using the mean vector $\boldsymbol{\mu} = [0.485, 0.456, 0.406]$ and standard deviation vector $\boldsymbol{\sigma} = [0.229, 0.224, 0.225]$, which correspond to the statistics of the ImageNet training set [32].

Class Imbalance Handling Chart image datasets inherently exhibit skewed class distributions, where certain chart types are far more prevalent than others. When such imbalance is left unaddressed, the model tends to be biased toward majority classes and performs poorly on rare ones [33]. To mitigate this, two complementary strategies are adopted.

First, each training sample is assigned a sampling weight inversely proportional to the frequency of its class, so that under-represented classes are drawn more frequently during mini-batch construction. This resampling procedure ensures that each class contributes more uniformly to the parameter updates across training iterations.

Second, the cross-entropy loss function is augmented with per-class weights to further penalise misclassifications of minority classes. Let N denote the total number of training samples, C the number of classes, and n_c the number of training samples belonging to class c. The weight assigned to class c is then defined as:

$$w_c = \frac{N}{C \cdot n_c} \tag{1}$$

By construction, classes with fewer samples receive higher weights, amplifying their contribution to the loss. The weighted cross-entropy loss for a prediction $\hat{y}$ given the ground-truth class c is thus:

$$\mathscr{L} = -w_c \log \hat{p}_c \tag{2}$$

where $\hat{p}_c$ is the predicted probability for the correct class c. Together, the two mechanisms direct the

optimiser's attention more equitably across all eight categories.

Two-Phase Training Strategy Training follows a two-phase transfer learning paradigm [29] designed to stabilise convergence and prevent catastrophic forgetting of the pre-trained representations.

In the first phase, the backbone parameters are frozen and only the newly added classification head is updated. The Adam optimiser [34], which adapts the learning rate individually for each parameter based on estimates of first- and second-order gradient moments, is used with a comparatively high learning rate η_{frozen}. This allows the classification head to converge rapidly toward the feature space already encoded in the frozen backbone, without risking interference with the pre-trained weights.

In the second phase, all model parameters are unfrozen and the entire network undergoes end-to-end fine-tuning with a smaller learning rate $\eta_{\text{finetune}} \ll \eta_{\text{frozen}}$. To anneal the learning rate smoothly throughout this phase, a cosine annealing schedule [35] is applied, which gradually reduces the learning rate from η_{finetune} to a minimum of $\eta_{\min} = 10^{-6}$ following a cosine curve. An optional linear warmup period [36] at the start of the second phase linearly increases the learning rate from a small initial value to η_{finetune} over several epochs, preventing destabilisingly large gradient updates on freshly unfrozen parameters.

To prevent overfitting, an early stopping criterion [37] is applied throughout training: if the validation accuracy does not improve for a predefined number of consecutive epochs, referred to as the patience, training is terminated early. The model checkpoint that achieved the highest validation accuracy is saved and subsequently restored for final evaluation.

Additionally, mixed-precision training [38] is employed on GPU hardware, wherein computations are performed in 16-bit floating-point precision where numerically safe and in 32-bit precision otherwise. This approach reduces memory consumption and accelerates training with negligible impact on model accuracy.

Evaluation Protocol The trained model is evaluated on a held-out test set that was not used during training or hyperparameter selection. Three metrics are reported: overall classification accuracy, the F1-score for each individual class, and the macro-averaged F1-score [39]. The F1-score for a given class is the harmonic mean of precision and recall for that class, and the macro-averaged F1-score is the unweighted mean of the per-class F1-scores, treating all classes equally regardless of their frequency. In addition, the confusion matrix is recorded to facilitate fine-grained analysis of inter-class misclassifications.

Training Results Figure 9 presents the training and validation loss and accuracy curves across all training epochs. The training loss decreases rapidly and consistently, approaching near-zero values by the final epochs. The validation loss exhibits greater oscillation in the early epochs, attributable to the transition between the two training phases and the inherent variance of the smaller validation set, before stabilising in the latter half of training. On the accuracy side, training accuracy converges to approximately 99%, while validation accuracy stabilises at around 95%, demonstrating that the model successfully learns discriminative features across all eight chart types without severe overfitting. The modest gap between training and validation accuracy is consistent with the expectations for a dataset of limited scale and reflects reasonable generalisation performance.

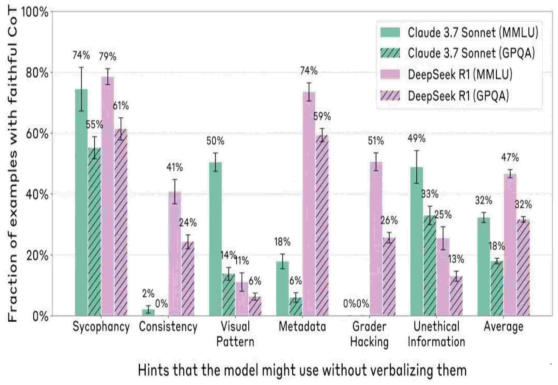
Figure 9: Training and validation loss (left) and accuracy (right) of ResNet-18 over the course of 8-class chart classification training. The training loss decreases steadily throughout all epochs, while the validation loss exhibits early oscillations corresponding to the transition between the frozen-backbone and full fine-tuning phases before converging. Training accuracy reaches approximately 99% and validation accuracy stabilises at approximately 95%, indicating effective generalisation across all eight chart categories.

4.4.2 Chart Parsing to Structured Markdown

Although VinternVL, the vision-language model used in our pipeline, is equipped with a visual encoder capable of directly perceiving chart images, relying solely on visual perception for numerical reasoning introduces a fundamental limitation in the context of Chart QA. Many chart related questions require precise retrieval of specific values, such as the exact percentage of a category, the highest or lowest data point, or the difference between two series. Vision encoders excel at capturing global visual patterns such as shapes, colours, and spatial layouts, but are prone to imprecision when extracting exact numerical values from visual encodings such as bar heights, pie sector angles, or line positions [6]. Small differences in value that are visually indistinguishable can nevertheless lead to incorrect answers on data-intensive questions. To address this, we introduce an intermediate parsing step that converts each chart image into a Markdown table using PaddleOCR-VL, providing VinternVL with an explicit, unambiguous textual representation of the chart's underlying data alongside the image itself.

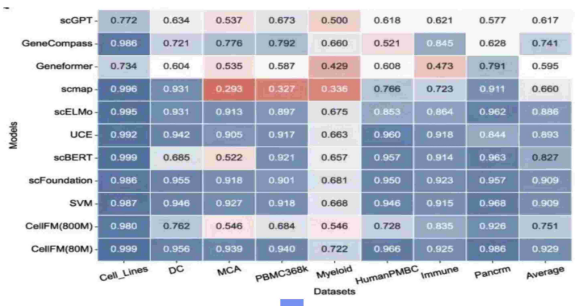
The resulting Markdown table is combined with the original chart image and the user's question to form a multimodal input prompt for VinternVL. This design allows VinternVL to leverage both modalities simultaneously for Chart QA: the visual input provides contextual and structural cues such as chart type, colour encoding, and overall trend, while the Markdown table supplies the precise numerical values and labels that the language component needs to answer questions accurately. By grounding numerical reasoning in structured text rather than visual inference alone, this approach significantly reduces the risk of hallucination on data-intensive questions, while preserving the visual context necessary for questions about patterns, comparisons, or overall chart structure [6].

Two representative examples of the parsing output are illustrated in Figures 10a and 10b. In each case, the upper portion shows the original chart image and the lower portion presents the corresponding structured table produced by PaddleOCR-VL, which faithfully preserves all row and column labels together with their associated numerical values.



Hints that the model might use without verbalizing them	Claude 3.7 Sonnet (MMLU)	Claude 3.7 Sonnet (GPQA)	DeepSeek R1 (MMLU)	DeepSeek R1 (GPQA)
Sycophancy	74%	55%	79%	61%
Consistency	2%	0%	41%	24%
Visual Pattern	50%	14%	11%	6%
Metadata	18%	6%	74%	59%
Grader Hacking	0%	0%	51%	26%
Unethical Information	49%	33%	25%	13%
Average	32%	18%	47%	32%

(a) Parsing example for a grouped bar chart: the original chart (top) is converted into a structured table (bottom) that encodes all category labels and percentage values across each model and benchmark combination.



Models	Cell_Lines	DC	MCA	PBMC368k	Myeloid	HumanPMBC	Immune	Pancrm	Average
scGPT	0.772	0.634	0.537	0.673	0.500	0.618	0.621	0.577	0.617
GeneCompass	0.986	0.721	0.776	0.792	0.660	0.521	0.845	0.628	0.741
Geneformer	0.734	0.604	0.535	0.587	0.429	0.608	0.473	0.791	0.595
scmap	0.996	0.931	0.293	0.327	0.336	0.766	0.723	0.911	0.660
scELMo	0.995	0.931	0.913	0.897	0.675	0.853	0.864	0.962	0.886
UCE	0.992	0.942	0.905	0.917	0.663	0.960	0.918	0.844	0.893
scBERT	0.999	0.685	0.522	0.921	0.657	0.957	0.914	0.963	0.827
scFoundation	0.986	0.955	0.918	0.901	0.681	0.950	0.923	0.957	0.909
SVM	0.987	0.946	0.927	0.918	0.668	0.946	0.915	0.968	0.909
CellFM(800M)	0.980	0.762	0.546	0.684	0.546	0.728	0.835	0.926	0.751
CellFM(80M)	0.999	0.956	0.939	0.940	0.722	0.966	0.925	0.986	0.929

(b) Parsing example for a heatmap: the original heatmap (top) is converted into a structured table (bottom) that preserves all model names, dataset identifiers, and accuracy scores for downstream reasoning.

Figure 10: Examples of chart parsing by PaddleOCR-VL: each chart image (top) is converted into a structured Markdown table (bottom), providing explicit numerical data for VinternVL's Chart QA reasoning stage.

4.4.3 Reasoning (Fine-tuning Vintern-1B-v2)

Fine-Tuning Overview Fine-tuning adapts a pretrained model to a specific domain or task by continuing its training on a smaller, curated dataset. Rather than training the entire model from scratch, which requires enormous computational resources, fine-tuning starts from a model that already understands language and vision, and teaches it the particular patterns, vocabulary, and reasoning style of the target domain. In this work, the target domain is Vietnamese chart visual question answering.

Two fine-tuning strategies are commonly used, and the choice between them depends on available GPU resources. The first is **full fine-tuning**, where all model parameters are updated during training. This yields the highest potential performance but requires a large amount of GPU memory. The second is **LoRA fine-tuning**, where only a small set of additional lightweight matrices are trained while the original model weights remain frozen. This requires far less memory and is therefore practical on consumer-grade or cloud GPUs with limited VRAM. In this work, LoRA fine-tuning was used, as it fits within the constraints of a single 16 GB GPU.

Our Fine-Tuning Method Low-Rank Adaptation (LoRA) [40] is a parameter-efficient fine-tuning technique. Instead of retraining all of a model's parameters, LoRA attaches small additional weight matrices to specific layers and trains only those. This dramatically reduces the GPU memory and time required for fine-tuning. Once training is complete, these additional matrices are *merged* back into the original model weights, so that the final model behaves identically to a fully retrained one without

any extra computational overhead at inference time. Figure 11 illustrates this mechanism.

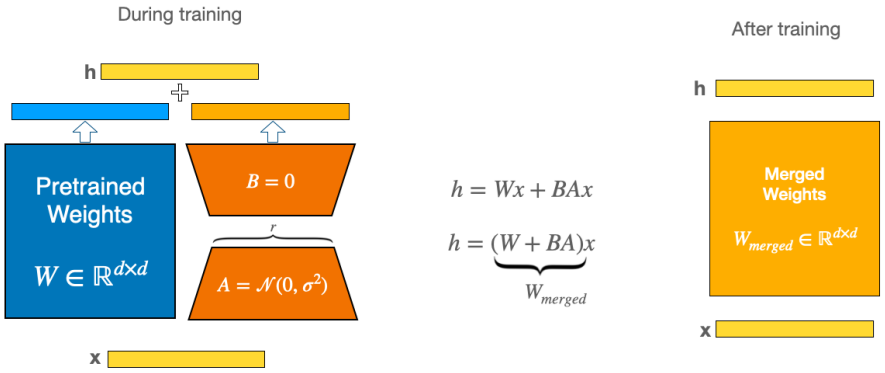


Figure 11: Illustration of the Low-Rank Adaptation (LoRA) mechanism, showing the parallel path of trainable decomposition matrices **A** and **B** alongside the frozen pre-trained weights.

Model Architecture Vintern-1B-v2 [41] follows the **ViT-MLP-LLM** paradigm, a three-component pipeline where a vision encoder, a projection layer, and a language model are connected in sequence [42, 43]. Each component is independently pretrained and then assembled:

Chart Image ⟶	`[Vision Encoder]`	⟶ `[MLP Projector]` ⟶	`[Language Model]`	⟶ Answer
	InternViT-300M ~300M params	projection bridges the two spaces	Qwen2-0.5B ~500M params	

The key advantage of this paradigm is modularity: fine-tuning can selectively update only the components that require domain adaptation, leaving the rest intact.

Vision Encoder The vision encoder used in this work is **InternViT-300M** [44], a compact Vision Transformer (ViT) [45] distilled from the much larger InternViT-6B via knowledge distillation [46]. Despite its small size (~300M parameters), it inherits strong visual representations suitable for dense visual tasks such as chart reading.

The encoder processes each image tile through the following stages:

1. **Patch Embedding:** The image tile (448×448 pixels) is divided into non-overlapping 14×14 pixel patches, yielding $(448/14)^2 = 1{,}024$ patches. Each patch is linearly projected into a feature vector and assigned a positional encoding.

2. **Transformer Encoder (24 stacked layers):** Each layer applies Layer Normalization, Multi-Head Self-Attention (every patch attends to every other patch), a residual connection, a second Layer Normalization, a Feed-Forward Network (two linear layers with GELU activation), and a final residual connection. Residual connections [45] prevent vanishing gradients and allow stable training of deep networks.

3. **Feature Selection:** Only the output of the final (24th) transformer layer is used. Earlier layers capture low-level features (edges, colors); the final layer captures the richest semantic content (chart axes, bars, labels).

4. **Pixel Unshuffle (spatial downsampling):** Inspired by sub-pixel convolution techniques [47], the token count is reduced by a factor of 4: 1,024 patches are rearranged into 256 visual tokens per tile. This operation trades spatial resolution for channel depth, preserving content while dramatically reducing sequence length.

The output is thus **256 feature vectors per tile**, each constituting one visual token.

MLP Projector The vision encoder and the language model operate in different embedding spaces, as their internal vector representations are not directly compatible. The MLP projector (a two-layer feed-forward network) serves as a **learned translation layer** between the two spaces [43, 42]:

$$\text{Visual tokens (ViT space)} \xrightarrow{\text{Linear+GELU}} \xrightarrow{\text{Linear}} \text{Projected tokens (LLM space)}$$

The GELU activation [45] enables the network to learn complex non-linear transformations. This projector was trained during Vintern-1B-v2's original pretraining phase, specifically to align the two modalities. It is *frozen* during fine-tuning to preserve this alignment.

Language Model The language model is **Qwen2-0.5B-Instruct** [48], a decoder-only Transformer that generates answer text autoregressively, one token at a time, by attending to all preceding tokens (both visual and textual).

Two architectural features are particularly relevant to this fine-tuning:

Grouped-Query Attention (GQA) [49] The standard multi-head attention uses one set of Query, Key, and Value matrices per attention head. GQA instead shares Key and Value matrices across groups of Query heads. With 14 Query heads but only 2 Key-Value head groups, the model reduces its memory footprint significantly, which is critical when processing long sequences that include many visual tokens.

SwiGLU Feed-Forward Network [50] Each transformer layer contains a feed-forward sub-network with a gated activation function. The gate learns to suppress irrelevant features and amplify relevant ones, which is beneficial when the model must selectively attend to specific chart elements (e.g., axis values) while ignoring background noise.

Dynamic Image Tiling Chart images arrive in arbitrary aspect ratios, covering wide bar charts, tall line charts, and square pie charts. Naively resizing all images to a fixed 448×448 square would distort proportions and destroy fine-grained content such as axis labels and data values. To address this, the model uses **dynamic tiling** [42]: the image is divided into multiple 448×448 tiles whose arrangement is chosen to best preserve the original aspect ratio.

Tiling Procedure

1. Compute the image's aspect ratio (width / height).
2. Enumerate all valid tile grid configurations (rows $\times$ columns) such that the total number of tiles does not exceed the allowed maximum (e.g., 1×1, 1×2, 2×1, 2×2, 2×3, 3×2, $\ldots$).
3. Select the configuration whose aspect ratio (cols / rows) most closely matches the image's aspect ratio.
4. Resize the image to exactly fit the selected tile grid. Example: a 2×3 grid $\Rightarrow$ resize to $(2 \times 448) \times (3 \times 448) = 896 \times 1344$ pixels.
5. Crop the resized image into individual 448×448 tiles.
6. Append one additional *thumbnail* tile: the full image downscaled to 448×448, giving the model a global overview alongside local tile detail.

As a worked example, for a wide bar chart with original size 800×400 (aspect ratio = 2.0), the best-matching configuration is 2 columns × 1 row (aspect ratio = 2.0, difference = 0.0). The image is resized to 896×448, cropped into 2 tiles, and a thumbnail is appended, yielding 3 tiles and $3 \times 256 = 768$ visual tokens.

Token Budget The total number of visual tokens grows linearly with the number of tiles (256 tokens per tile). During training, the maximum number of tiles was limited to 6 (plus thumbnail), giving up to $7 \times 256 = 1{,}792$ visual tokens, in order to stay within GPU memory constraints. At inference time, this limit is raised to 12 (plus thumbnail), allowing the model to process finer-grained chart details at up to $13 \times 256 = 3{,}328$ visual tokens.

Freezing Strategy

The Concept of Freezing In deep learning, **freezing** a module means setting its parameters to be non-trainable: no gradients are computed for them, and they are never updated by the optimizer. The frozen module still performs its full computation during every forward pass; it simply does not learn. This technique is widely used in transfer learning to preserve pretrained representations while adapting only a subset of the model [43, 42].

What Is Frozen and Why The parameter status across components is summarized in Table 2.

Table 2: Freeze strategy overview.

Component	Status	Reason
Vision Encoder (~300M params)	Frozen	Pretrained to extract rich visual features; retraining would require far more GPU memory and risks degrading learned representations.
MLP Projector (few M params)	Frozen	Cross-modal alignment was carefully learned during pretraining; unfreezing with both ends frozen provides little additional benefit.
Language Model base weights (~500M params)	Frozen	Preserves general Vietnamese language understanding and instruction-following ability.
LoRA Adapters (~2 to 5M params)	**Trainable**	The only parameters updated during training.

In total, only $\approx 3\text{M}/1{,}000\text{M} \approx 0.3\%$ of all parameters are trainable.

Forward Pass vs. Backward Pass Under Freezing The most important distinction: **freezing does not disable a module during training.** Every training step processes the full chart image through the full vision encoder and the full language model base weights. What freezing prevents is those modules from *changing* their behavior. During the backward pass, gradients flow only to the LoRA adapters and are blocked at all frozen module boundaries.

The practical implication is that the LoRA adapters learn to better interpret the fixed visual features that the frozen vision encoder produces. They cannot improve those features, as the ceiling on visual

understanding is set by the pretrained encoder. What they learn is: given these fixed visual features and this question, how should the answer distribution be adjusted for Vietnamese chart question answering?

LoRA Adapter Design

The Problem LoRA Solves Full fine-tuning of the language model's ~500M parameters would require storing a copy of all 500M parameters in GPU memory plus two additional 500M-parameter tensors for the optimizer's momentum estimates, which exceeds the memory capacity of commodity GPUs. **LoRA (Low-Rank Adaptation)** [40] solves this by introducing a small number of new trainable parameters arranged in a mathematically efficient structure, while keeping the original parameters frozen. Implementation is handled via the PEFT library [51].

How LoRA Works For each weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ in the language model, LoRA [40] adds a parallel branch consisting of two small matrices $\mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}$ and $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}$:

$$\mathbf{h} = \underbrace{\mathbf{W}\mathbf{x}}_{\text{frozen}} + \underbrace{\mathbf{B}\mathbf{A}\mathbf{x} \cdot \frac{\alpha}{r}}_{\text{trainable}}$$

where $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input vector, $\mathbf{h} \in \mathbb{R}^{d_{\text{out}}}$ is the output vector, $r = 16$ is the rank (the bottleneck dimension controlling the expressiveness of the adapter), $\alpha = 32$ is a scaling hyperparameter, and d_{in}, d_{out} are the input and output dimensions of the original weight matrix respectively. The product $\mathbf{BA}$ is a **low-rank approximation** of the weight change $\Delta\mathbf{W}$ that full fine-tuning would have learned. It cannot represent arbitrary changes to $\mathbf{W}$, but it can capture the most important directions of adaptation, which is sufficient for a domain-specific task like Vietnamese chart question answering.

Initialization At the start of training, matrix $\mathbf{B}$ is initialized to all zeros, so that $\mathbf{BA} = \mathbf{0}$ at step 0. The model therefore behaves identically to the pretrained checkpoint at initialization, ensuring no sudden disruption to learned representations at the first training step.

Where LoRA Is Applied LoRA adapters are injected into every attention and feed-forward sublayer of all 24 decoder layers in the language model: the Query, Key, Value, and Output projections in the self-attention sublayer, and the Gate, Up, and Down projections in the feed-forward sublayer. The vision encoder receives **no LoRA adapters**; its representations are entirely fixed.

Trainable Parameter Budget Table 3 summarizes the parameter counts. Only approximately 0.2% to 0.5% of all parameters are trainable, which is what makes single-GPU fine-tuning of a 1-billion-parameter multimodal model feasible.

Table 3: Parameter budget under the LoRA fine-tuning strategy.

Component	**Parameters**	**Status**
Vision encoder	~300,000,000	Frozen
MLP projector	~2,000,000	Frozen
Language model (base)	~500,000,000	Frozen
LoRA adapters	~2,000,000 to 5,000,000	**Trainable**
Total	~1,000,000,000	
Trainable / Total	≈ 0.2% to 0.5%	

Training Sequence Construction

Input Sequence Layout During training, each sample is formatted as a single token sequence that interleaves visual tokens and text tokens following the **Hermes-2 chat template**. The layout consists of three blocks: (1) a system block containing the system instruction; (2) a user block containing the visual tokens (one block of 256 tokens per tile, including the thumbnail tile), followed by the question text; and (3) an assistant block containing the target answer tokens. Each placeholder is replaced at runtime with the corresponding projected visual vector from the MLP projector. From the language model's perspective, visual tokens and text tokens are processed identically, as they are all vectors in the same embedding space [43].

Loss Masking The training loss is computed using **cross-entropy**, measuring how well the model predicts each answer token given all previous tokens. Crucially, only the assistant's answer tokens contribute to the loss; all other tokens (system prompt, visual tokens, question text) are masked:

$$\mathscr{L} = -\sum_{t \in \mathscr{A}} \log p_\theta\big(y_t \mid y_{<t}, \mathbf{v}, \mathbf{q}\big)$$

where $\mathscr{A}$ is the index set of answer token positions, y_t is the ground-truth token at position t, $y_{<t}$ denotes all preceding tokens (both visual and textual), $\mathbf{v}$ is the sequence of projected visual tokens produced by the MLP projector, $\mathbf{q}$ is the sequence of question tokens, θ denotes all trainable parameters (i.e., the LoRA adapters), and $p_\theta(y_t \mid y_{<t}, \mathbf{v}, \mathbf{q})$ is the probability the model assigns to the correct token y_t given all preceding context. This design ensures the model is trained to *answer questions* given visual context, not to describe images unconditionally [11, 52].

Training Configuration

Optimizer and Learning Rate Schedule Training uses the **AdamW optimizer** [53] with a cosine learning rate schedule [54] and a short linear warmup phase. AdamW applies decoupled weight decay directly to the weights rather than through the gradient, which is more stable for fine-tuning pretrained models. The learning rate increases linearly from zero to its peak value (4×10^{-5}) over the first 3% of training steps (warmup), then follows a cosine decay curve to near zero over the remaining 97%. The hyperparameter settings are summarized in Table 4.

Table 4: Training hyperparameters.

Hyperparameter	Value
Peak learning rate	4×10^{-5}
Weight decay	0.01
Scheduler	Cosine with linear warmup
Warmup proportion	3% of total steps
Training epochs	1

Memory Optimization Techniques Training a 1-billion-parameter multimodal model on a single commodity GPU requires several memory optimization strategies applied simultaneously.

LoRA (Optimizer Memory Reduction) By training only ~3M parameters instead of ~500M, the AdamW optimizer's momentum buffers shrink proportionally: from ≈4 GB (for full fine-tuning) to ≈24 MB, saving roughly 4 GB of GPU memory [40].

Gradient Checkpointing (Activation Memory Reduction) [55] Intermediate activations required for the backward pass are not stored during the forward pass. Instead, they are recomputed on demand during the backward pass. This trades approximately 30% extra compute time for a significant reduction in peak memory. Concretely, let L denote the number of transformer layers and S the sequence length. Without checkpointing, peak activation memory grows as $\Theta(L \cdot S)$ that is, proportionally to the product $L \cdot S$ because intermediate activations from all L layers must be retained in GPU memory simultaneously for use during the backward pass. With checkpointing, this is reduced to $\Theta(L^{1/2} \cdot S)$: only the activations at $\lfloor L^{1/2} \rfloor$ evenly spaced checkpoint layers are retained; the activations of the remaining layers are discarded and recomputed on demand during the backward pass. Here $\Theta(\cdot)$ denotes tight asymptotic growth, meaning memory scales exactly at this rate (up to a constant factor) rather than merely being bounded above by it.

Mixed Precision (bfloat16) (Weight and Activation Memory Reduction) [56] All model weights and activations are stored in `bfloat16` format during training. This halves memory use compared to `float32` (~2 GB vs. ~4 GB for 1B parameters). `bfloat16` is preferred over `float16` because its wider exponent range avoids gradient underflow issues in deep networks.

Sequence Length Limit (Sequence Memory Reduction) The total sequence length (visual tokens plus text tokens) is capped at 700 tokens during training, directly limiting peak activation memory per sample. The language model's native context length is 4,096 tokens, nearly 6× longer. Reducing to 700 enables single GPU training but means samples with many tiles or long answers may be truncated.

Length-Grouped Batching (Training Efficiency) Samples are grouped so that those with similar sequence lengths appear in the same batch. This minimizes padding overhead and improves the fraction of each batch's compute that contributes meaningfully to training.

Checkpoint Resumption The training script is adapted from the official InternVL 2.0 LoRA fine-tuning configuration for the 1B-scale model [42]. Unlike approaches that resume from existing checkpoints, this training was conducted from scratch using a freshly initialized LoRA adapter, where the adapter matrices **A** and **B** were assigned random weights. The adapter was trained for a total of 44,671 steps on a combination of Vietnamese chart VQA data and self-constructed data, completing one full epoch. A peak learning rate of 4×10^{-5} was employed, which is a standard setting for fresh LoRA initialization to ensure stable convergence throughout the training process.

LoRA Merging

The Merge Operation Upon completion of the 44,671-step fine-tuning process, the LoRA adapter matrices are **permanently merged** into the frozen base weights of Vintern-1B-v2, producing a single self contained model file [40]:

$$\mathbf{W}_{\text{merged}} = \mathbf{W}_{\text{frozen}} + \mathbf{B} \cdot \mathbf{A} \cdot \frac{\alpha}{r}$$

where $\mathbf{W}_{\text{frozen}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the original pretrained weight matrix, $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}$ are the trained LoRA adapter matrices, α is the scaling hyperparameter, and r is the rank. The mathematical result is identical: the merged model produces exactly the same outputs as the LoRA model for any input. The merge is purely a structural simplification that eliminates the separate **A** and **B** matrices.

Why Merge? After merging, the model can be loaded and used as a standard HuggingFace `AutoModel` [57] with no special dependencies (e.g., the PEFT library [51]). Inference is faster since only one matrix multiplication per layer ($\mathbf{W}_{\text{merged}} \cdot \mathbf{x}$) is needed instead of two separate operations

($\mathbf{W} \cdot \mathbf{x} + \mathbf{BA} \cdot \mathbf{x}$), where $\mathbf{x}$ is the input activation vector at that layer. The trade-off is a larger file on disk compared to the compact LoRA checkpoint.

Training Loss Curve During fine tuning, the model training loss - a scalar value measuring the discrepancy between the model predicted token probabilities and the ground truth answer tokens via the cross entropy objective as described in the Loss Masking paragraph - was recorded at every optimization step. A lower loss indicates that the model assigns higher probability to the correct answer tokens, meaning its predictions are closer to the ground truth. Because the raw per step loss fluctuates due to batch level stochasticity in gradient estimation, a rolling average over 50 consecutive steps was computed to reveal the underlying trend more clearly.

Figure 12 illustrates the training loss trajectory over the full 44,671 step fine tuning process, corresponding to one complete epoch over the training data. The smoothed curve demonstrates a textbook exponential decay pattern, decreasing steadily from above 1.0 in the early steps and plateauing near 0.58 towards the final steps. While the raw per step loss exhibits expected variance arising from batch level optimization, the overall convergence of the smoothed curve confirms that the model successfully minimized the cross entropy objective and reached a stable optimization state.

Figure 12: Training loss curve of the Vintern-1B-v2 LoRA fine tuning run. The red line shows the smoothed 50 step rolling average; the blue line shows the raw per step loss. The steady decrease and eventual plateau near 0.58 indicate stable convergence.

Design Decisions and Trade-offs

Freeze Strategy Comparison The chosen strategy, applying LoRA on the language model with everything else frozen, occupies the most conservative position on the spectrum of fine-tuning approaches, as shown in Table 5. It sacrifices visual adaptation (the vision encoder cannot learn to better extract features specific to Vietnamese chart styles) in exchange for minimal memory use and training stability. Catastrophic forgetting [40], the tendency to overwrite previously learned knowledge, is largely eliminated because the original weights are never modified.

LoRA Rank Selection The rank $r = 16$ determines the expressiveness of the LoRA adaptation [40]. Lower ranks (e.g., $r = 8$, ~1.5M trainable params) are less expressive and may underfit, while higher ranks (e.g., $r = 32$, ~6M params) double the optimizer memory. Rank 16 is the standard recommendation for models at this scale and represents a well-validated balance between adaptation capacity and resource cost.

Table 5: Comparison of fine-tuning strategies. The chosen approach is highlighted in bold.

Strategy	**Train. params**	**GPU mem.**	**Forgetting**	**Visual adapt.**
Full fine-tune (all)	~1B	Very high	High	Complete
LLM only (no LoRA)	~500M	High	Medium	None
MLP + LLM	~502M	High	Medium	Indirect
LoRA on LLM, all frozen	**~3M**	**Low**	**Low**	**None**
LoRA on LLM, last ViT layers unfrozen	~15M	Medium	Low	Partial

Sequence Length Limit. Capping the sequence at 700 tokens is the most consequential constraint in this experiment. Given that visual tokens alone can occupy up to 1,792 positions (when using 6 tiles plus a thumbnail), many training samples will be truncated. In practice, most training samples use 1 to 3 tiles to remain within the budget; charts requiring 4 to 6 tiles for full resolution are processed at reduced detail. The trade-off is clear: fitting within single-GPU memory requires sacrificing the model's ability to fully leverage the dynamic tiling system's high-resolution capability during training.

5 System Design and Implementation

The proposed Chart Question Answering (CQA) system adopts a multi-stage pipeline architecture that decomposes the problem into a sequence of specialised modules, each responsible for a distinct processing step. Rather than relying on a single end-to-end model to handle all aspects of chart understanding, this decoupled design allows each component to be optimised independently and ensures that computationally intensive operations are only invoked when necessary. The overall architecture of the system is illustrated in Figure 13.

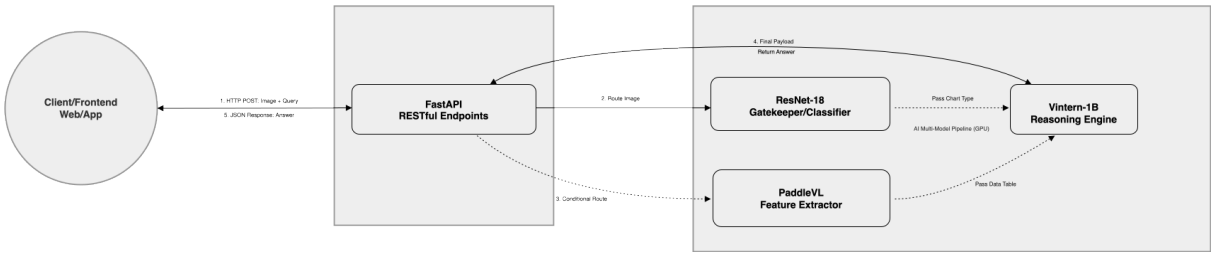


Figure 13: Overview of the proposed CQA system, showing the interaction between the client, the web server backend, and the three AI modules: the ResNet-18 classifier, the PaddleVL data extractor, and the Vintern-1B reasoning engine.

5.1 AI Model Integration

The three AI models in the pipeline are integrated through a centralised orchestration layer, which manages data exchange between components and enforces the conditional execution logic. Each model operates as an independent processing unit with a well-defined input/output contract, enabling them to be replaced or updated without affecting the other stages.

The **ResNet-18 classifier** receives a preprocessed image and returns a predicted chart-type label c^*. This label is passed to the pipeline controller, which uses it to decide whether to proceed with the remaining stages or to return an early error response to the user.

The **PaddleOCR-VL data extractor** runs in an isolated software environment and communicates with the main system through an internal network interface. The orchestration layer sends the chart image to this component and receives a structured Markdown table D_{table} in return. This separation cleanly decouples the two software environments and avoids dependency conflicts between PaddleOCR-VL and Vintern-1B.

The **Vintern-1B reasoning module** receives the original chart image, the Markdown table, the chart type label, and the user question as a unified multimodal prompt. It returns a free-form natural language answer, which is forwarded back to the web server for delivery to the user.

5.2 Data Flow and Processing

The data flow through the system proceeds in five sequential stages, as illustrated in Figure 14.

Stage 1: Input ingestion The user submits a chart image I and a natural language question Q through the system interface. The web server validates the request and forwards both inputs to the pipeline.

Stage 2: Preprocessing and classification The image is resized and normalised before being passed to the ResNet-18 classifier f_{cls}, which predicts the chart type:

$$c^* = \arg\max_{c \in \mathscr{C}} f_{\text{cls}}(I) \tag{3}$$

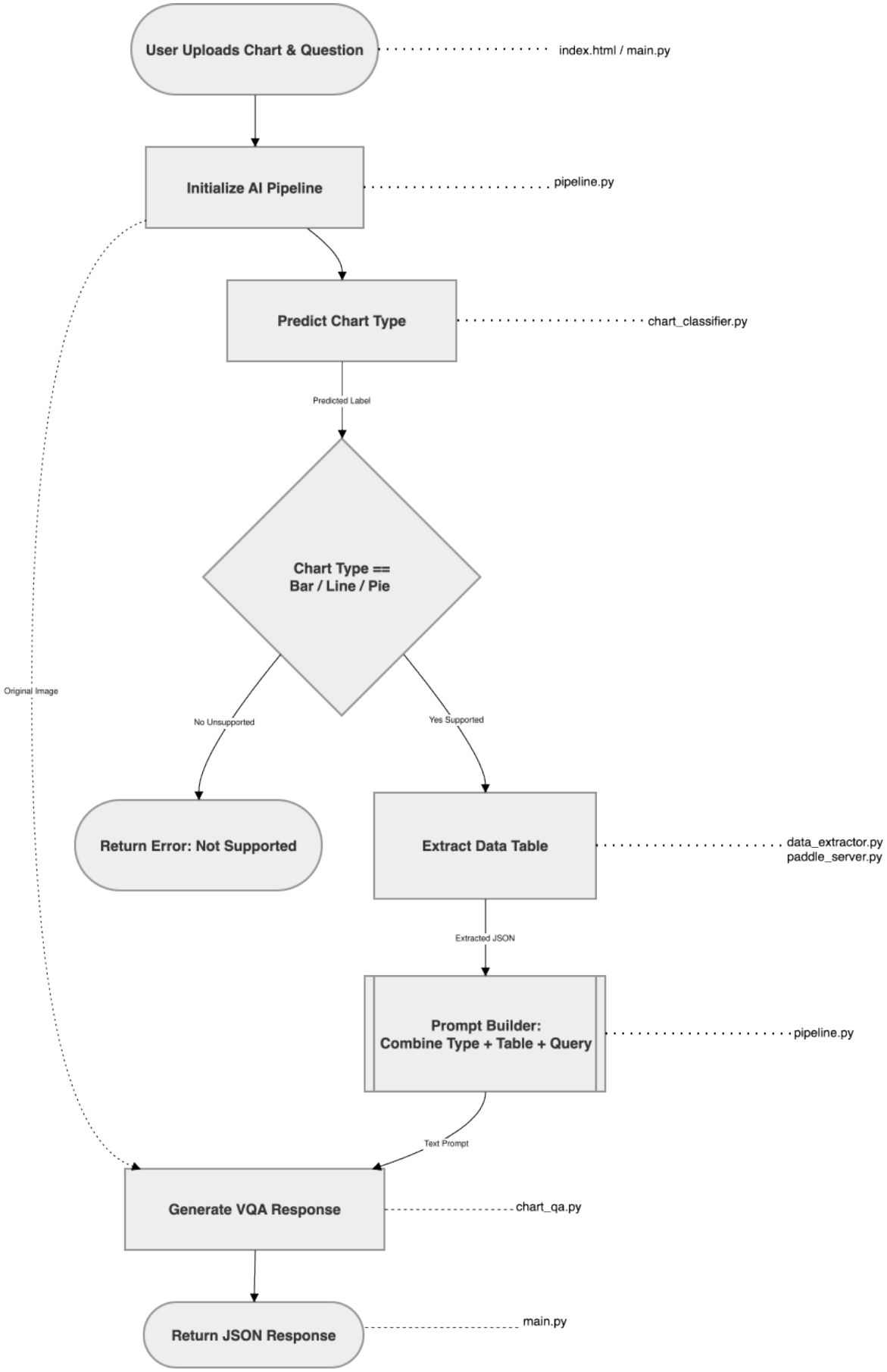


Figure 14: Execution flowchart of the CQA pipeline, illustrating the preprocessing step, the ResNet-18 classification and conditional routing decision, the PaddleVL data extraction, the prompt construction, and the Vintern-1B answer generation stages.

where $\mathscr{C}$ is the full set of recognisable chart classes and $\mathscr{S} = \{\text{Bar, Line, Pie}\}$ is the supported subset. If $c^* \notin \mathscr{S}$, the pipeline returns an error response immediately and no further processing occurs.

Stage 3: Data extraction If $c^* \in \mathscr{S}$, the image is forwarded to PaddleOCR-VL, which parses the chart's visual elements and produces a structured Markdown table:

$$D_{\text{table}} = f_{\text{ext}}(I) \tag{4}$$

This table explicitly encodes the axis labels, category names, and numerical values present in the chart, making the underlying data directly readable by the downstream language model.

Stage 4: Prompt construction and reasoning. The orchestration layer constructs a multimodal prompt by combining c^*, D_{table}, and Q. This prompt, alongside the original image I, is passed to the Vintern-1B reasoning module f_{vqa}, which generates the final answer:

$$A = f_{\text{vqa}}\big(I,\, D_{\text{table}},\, Q,\, c^*\big) \tag{5}$$

Stage 5: Response delivery. The web server assembles a structured response containing the answer A, the predicted chart type c^*, the extracted data table D_{table}, and per-stage processing time measurements, which is returned to the user.

5.3 Interface Design

The system provides two interfaces: a programmatic interface for machine clients and a web-based interface for human users.

The **programmatic interface** follows standard RESTful conventions. It accepts a request containing the chart image and the natural language question, and returns a structured response that includes the generated answer, the detected chart type, the extracted data table, and per-stage processing time measurements. This interface is intended for integration with other applications or automated evaluation pipelines.

The **web-based interface** allows human users to upload a chart image, type a question in natural language, and view the generated answer. Alongside the final answer, the interface also displays the detected chart type and the extracted data table, giving users visibility into the intermediate processing steps. This transparency allows users to verify the correctness of the data extraction before interpreting the answer, and provides useful diagnostic information when the system produces unexpected responses.

5.4 Deployment Strategy

A key deployment challenge is the dependency version conflict between PaddleOCR-VL and Vintern-1B, which require incompatible versions of the same underlying deep learning library. The system resolves this by adopting a two-environment deployment strategy, where each model group runs in its own isolated software environment:

1. **Primary environment:** Contains the web server, the ResNet-18 classifier, and Vintern-1B. This environment is started first and handles all incoming user requests.

2. **Extraction service environment:** Contains PaddleOCR-VL, which runs as an independent service and communicates with the primary environment through an internal network interface. It is started separately and its lifecycle is managed independently of the main server.

Both environments are deployed on an NVIDIA GPU-accelerated local workstation with CUDA support to accelerate model inference. The deployment procedure follows a simple two-step startup:

the extraction service is launched first, followed by the main web server. Once both services are running, the system is ready to accept user requests.

5.5 Scalability and Maintenance

The modular, pipeline-based architecture of the system facilitates both long-term maintenance and future scaling.

Scalability. Each stage of the pipeline operates independently and communicates through well-defined interfaces, making it straightforward to scale individual components. For instance, the data extraction service can be replicated and placed behind a load balancer if it becomes a throughput bottleneck, without requiring any modification to the classification or reasoning components. Extending the system to support additional chart types requires only retraining the ResNet-18 classifier on an expanded dataset and updating the routing logic in the orchestration layer; the extraction and reasoning modules remain unchanged. The two-service architecture also provides a natural path toward containerisation using technologies such as Docker, enabling future deployment on cloud platforms for horizontal scaling.

Maintenance and version management. The two environment strategy isolates dependency changes so that updating one model does not affect the other. Each model can be independently versioned and replaced, provided the input/output interface contract is preserved. Per stage processing time measurements included in every response provide continuous observability into system performance, enabling early detection of regressions or bottlenecks following updates. Should a more capable vision language model become available, it can be integrated into the reasoning stage with no impact on the classification or extraction components.

6 Results and Discussion

6.1 Results and Analysis

This section presents the evaluation results for the two core components of the proposed CQA system, which are the ResNet-18 chart classification module and the Vintern-1B vision language model after fine tuning. The classification module is evaluated using standard classification metrics, while the question answering module is evaluated using text generation metrics commonly adopted in the chart QA literature.

6.1.1 Chart Classification

Evaluation Metrics To quantitatively assess the performance of the proposed chart classifier, four standard metrics widely used in the pattern recognition and information retrieval literature were adopted, namely precision, recall, F1 score, and overall accuracy [58, 59].

Given a multiclass classification problem, let TP_c, FP_c, and FN_c denote the number of true positives, false positives, and false negatives for class c, respectively. The per class metrics are defined as:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \tag{6}$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \tag{7}$$

$$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{8}$$

where Precision (Eq. 6) measures the fraction of predicted instances of class c that are genuinely of that class, penalising false alarms. Recall (Eq. 7) measures the fraction of true instances of class c that the model successfully retrieves, penalising missed detections. The F1 score (Eq. 8) is the harmonic mean of precision and recall, providing a single balanced indicator that is more informative than accuracy alone when class sizes are unequal [59].

Overall accuracy is defined as the ratio of correctly classified samples to the total number of samples:

$$\text{Accuracy} = \frac{\sum_{c=1}^{C} TP_c}{N} \tag{9}$$

where C is the number of classes and N is the total number of test samples. For a holistic view across all classes, the macro averaged F1 score is computed as the unweighted mean of per class F1 scores:

$$\overline{\text{F1}} = \frac{1}{C} \sum_{c=1}^{C} \text{F1}_c \tag{10}$$

Macro averaging treats every class equally regardless of its support size, making it particularly suitable for balanced evaluation in multi class settings [60]. Accuracy provides an intuitive global measure of

model performance. However, it can be misleading when the class distribution is highly imbalanced, since a classifier that always predicts the majority class may still achieve high accuracy [60].

The macro averaged F1 score assigns equal weight to every class regardless of its support size. This makes it especially informative when the practitioner wants to ensure that the model does not sacrifice performance on minority classes in favour of majority ones [60]. In contrast, the weighted average F1 score scales each per class F1 score by the proportion of samples belonging to that class:

$$\overline{\mathrm{F1}}_{\mathrm{weighted}} = \frac{1}{N} \sum_{c=1}^{C} n_c \cdot \mathrm{F1}_c \tag{11}$$

where n_c is the number of true instances of class c and $N = \sum_{c=1}^{C} n_c$ is the total number of test samples. Weighted averaging reflects the overall performance as experienced in practice, giving more influence to classes that appear more frequently in the dataset. When the dataset is perfectly balanced, meaning n_c is identical for all c, as is the case in our evaluation with 76 samples per class, the macro averaged and weighted averaged scores coincide [60], which is confirmed by the identical values reported in Table 6.

Results The ResNet-18 classifier was evaluated on a held out test set of 608 samples with 76 samples per class, collected independently from the training dataset.

Table 6: Classification report of the proposed model on the test set.

Class	**Precision**	**Recall**	**F1 score**	**Support**
Area	0.9722	0.9211	0.9459	76
Bar	0.8625	0.9079	0.8846	76
Box	0.9861	0.9342	0.9595	76
Heatmap	0.8750	0.9211	0.8974	76
Histogram	1.0000	0.9211	0.9589	76
Line	0.8161	0.9342	0.8712	76
Pie	1.0000	1.0000	1.0000	76
Scatter	0.9155	0.8553	0.8844	76
Accuracy		0.9243		608
Macro avg	0.9284	0.9243	0.9252	608
Weighted avg	0.9284	0.9243	0.9252	608

The classifier achieves an overall accuracy of **92.43%** across all eight chart categories. A detailed analysis of Table 6 shows that the Pie chart class achieves a perfect F1 score of 1.0000, indicating flawless classification for this category. Both Histogram and Box charts also perform exceptionally well, with precision above 98% and F1 scores around 0.96. On the other end of the spectrum, Line charts present the most challenge for the model, recording the lowest precision at 0.8161 and F1 score at 0.8712. This suggests some confusion with other chart types, likely due to visual similarity when using small data points or specific aspect ratios. Scatter plots record the lowest recall at 0.8553, indicating that some scatter plots were misidentified.

Since the macro average and weighted average are identical across all metrics, this confirms that the test set is perfectly balanced across all eight classes, and the reported macro figures accurately represent the generalized performance of the model without class imbalance effects.

6.1.2 Chart Question Answering

Evaluation Metrics The question answering module was evaluated using six automatic text generation metrics: BLEU, METEOR, ROUGE 1, ROUGE 2, ROUGE L, and BERTScore [61, 62, 63, 64]. These metrics collectively assess lexical overlap, n gram precision and recall, and semantic similarity between the generated answers and the reference answers.

BLEU The Bilingual Evaluation Understudy (BLEU) score measures the geometric mean of modified n gram precision scores between a candidate sentence and one or more reference sentences, with a brevity penalty applied to discourage overly short outputs:

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{12}$$

where N is the maximum n gram order considered, p_n is the modified n gram precision of order n, and BP is the brevity penalty defined as:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases} \tag{13}$$

The brevity penalty equals 1 when the candidate is at least as long as the reference and decays exponentially when the candidate is shorter. BLEU ranges from 0 to 1, where higher values indicate greater lexical overlap with the reference.

METEOR The Metric for Evaluation of Translation with Explicit Ordering (METEOR) addresses several limitations of BLEU by incorporating stemming, synonym matching, and a chunk based fragmentation penalty. It is computed as:

$$\text{METEOR} = (1 - \gamma \cdot \text{frag}^{\beta}) \cdot \frac{P \cdot R}{\alpha P + (1-\alpha) R} \tag{14}$$

where P is the unigram precision, R is the unigram recall, and frag is the fragmentation penalty. A higher value of frag indicates that matches are scattered and non contiguous, reflecting poor word ordering.

ROUGE Recall Oriented Understudy for Gisting Evaluation (ROUGE) is a family of metrics originally designed for summarisation evaluation. Three variants are used in this work. ROUGE 1 measures the overlap of unigrams, ROUGE 2 measures the overlap of bigrams, and ROUGE L measures the longest common subsequence (LCS). ROUGE L replaces n gram counting with an LCS based F1 score:

$$\text{ROUGE-L} = \frac{(1+\beta^2) \cdot R_{\text{lcs}} \cdot P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 \cdot P_{\text{lcs}}} \tag{15}$$

where LCS does not require matches to be contiguous, making it sensitive to sentence level word order.

BERTScore Unlike the lexical metrics above, BERTScore leverages contextual embeddings from a pre trained BERT model to measure semantic similarity at the token level. For every token in the

candidate, it finds the most similar token in the reference via cosine similarity, and vice versa:

$$F_{\text{BERT}} = \frac{2 \cdot P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{16}$$

BERTScore captures paraphrastic and semantic equivalences that purely lexical metrics cannot detect, making it a more robust evaluation criterion for open ended generation tasks.

Results Figure 15 presents the metric scores for Vintern-1B before and after fine tuning on the CQA dataset. Fine tuning consistently improves performance across all six metrics, confirming that the domain specific training data effectively adapts the language generation of the model to the chart question answering task.

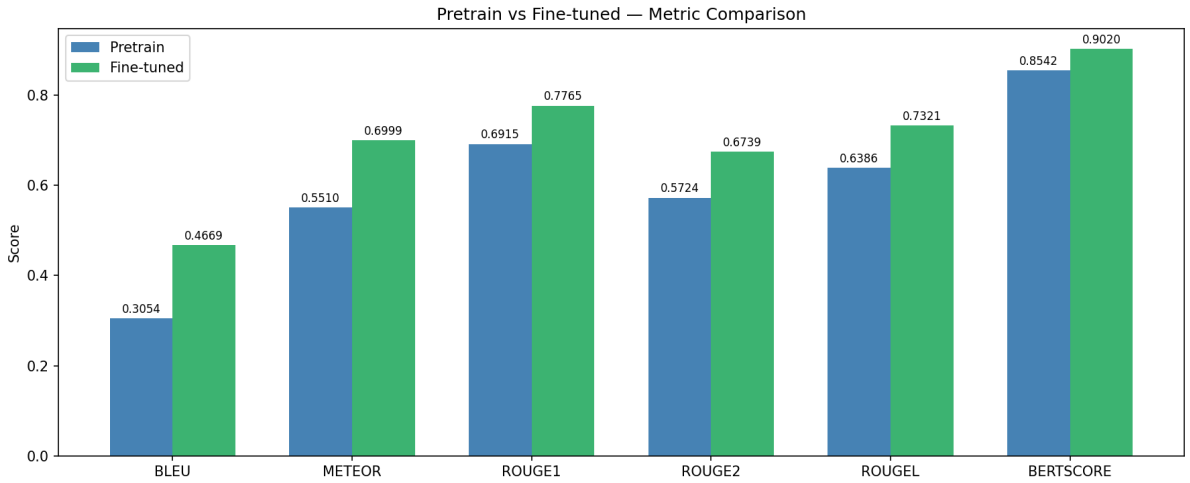


Figure 15: Comparison of Vintern-1B before and after fine tuning across all six evaluation metrics on the CQA test set.

Figure 16 further compares the fine tuned Vintern-1B against five baseline language models, which are Gemma-2-2B [65], Llama-3.2-1B-Instruct [66], Qwen-2.5-VL-2B [67], SeaLLMs-v3-1.5B [68], and StableLM-1.6B [69] across all evaluation metrics.

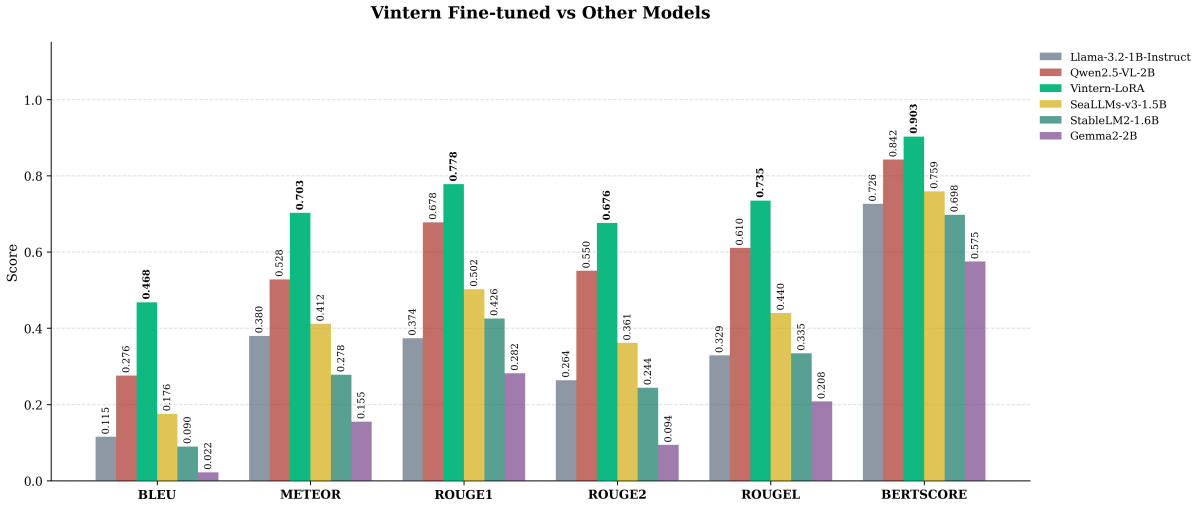


Figure 16: Metric comparison of the fine tuned Vintern-1B against five baseline language models on the CQA test set.

To further assess the generalisability of the fine tuning approach described in Section 4.4.3 and the dataset introduced in the Reasoning Dataset paragraph, the same training procedure and data are applied to two additional vision language models that are architecturally comparable to Vintern-1B-v2. Both candidate models share the same two component design as Vintern-1B-v2, including a visual encoder that extracts image level representations and a language model that generates textual responses conditioned on those representations.

The two models selected for this comparison are InternVL2-1B and Qwen2-VL-2B [67]. Both are fine tuned using parameter efficient adaptation methods to keep training costs comparable to those of Vintern-LoRA. Specifically, InternVL2-1B is fine tuned using Quantised Low Rank Adaptation (QLoRA) [70], which combines 4 bit weight quantisation with low rank gradient updates to drastically reduce memory requirements without sacrificing adaptation capacity. Qwen2-VL-2B is fine tuned using Low Rank Adaptation (LoRA) [71], which constrains weight updates to low rank decompositions of the original weight matrices. Both methods are applied under the same hyperparameter configuration and training data as Vintern-LoRA, ensuring a fair and controlled comparison.

As shown in Figure 17, Vintern-LoRA outperforms InternVL2-1B across all metrics by a substantial margin. The most pronounced gap is observed in BLEU, where InternVL2-1B scores 0.253 against 0.468 for Vintern-LoRA, which is an absolute improvement of 0.215. Gains of similar magnitude are observed across the remaining metrics, with BERTScore rising from 0.837 to 0.903. The consistent gap across both lexical and semantic metrics indicates that InternVL2-1B, despite sharing a comparable architecture and receiving identical fine tuning supervision, does not adapt as effectively to the chart question answering domain.

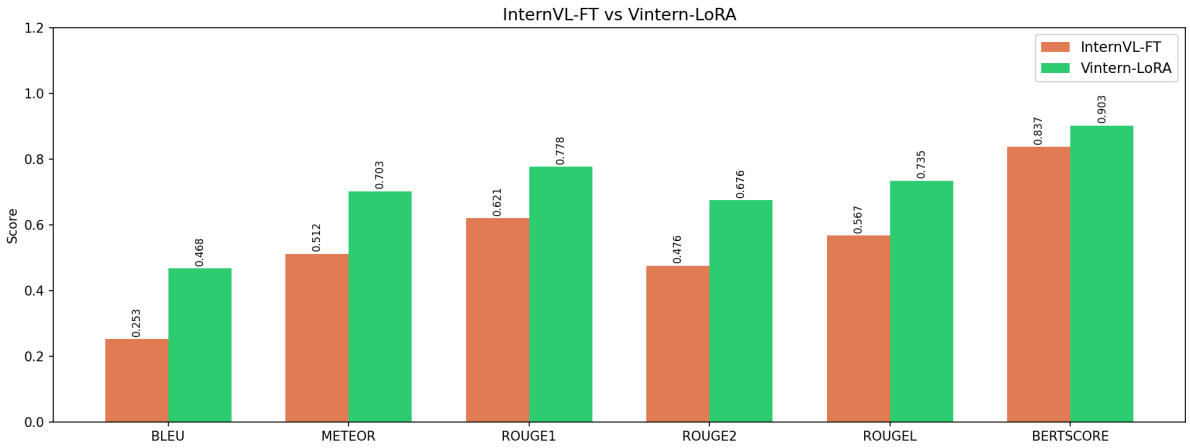


Figure 17: Metric comparison between InternVL2-1B (QLoRA) and Vintern-LoRA on the CQA test set. Vintern-LoRA achieves superior scores across all six metrics.

Figure 18 reveals a similar pattern for Qwen2-VL-2B. Despite having approximately twice the parameter count of Vintern-1B-v2, Qwen2-VL-2B nonetheless underperforms Vintern-LoRA across every metric, with BLEU at 0.239 versus 0.468 and BERTScore at 0.786 versus 0.903. It is further notable that Qwen2-VL-2B yields results largely comparable to those of InternVL2-1B, indicating that neither a larger parameter count nor the choice of adaptation method compensates for the domain alignment advantage that Vintern-1B-v2 brings to this task.

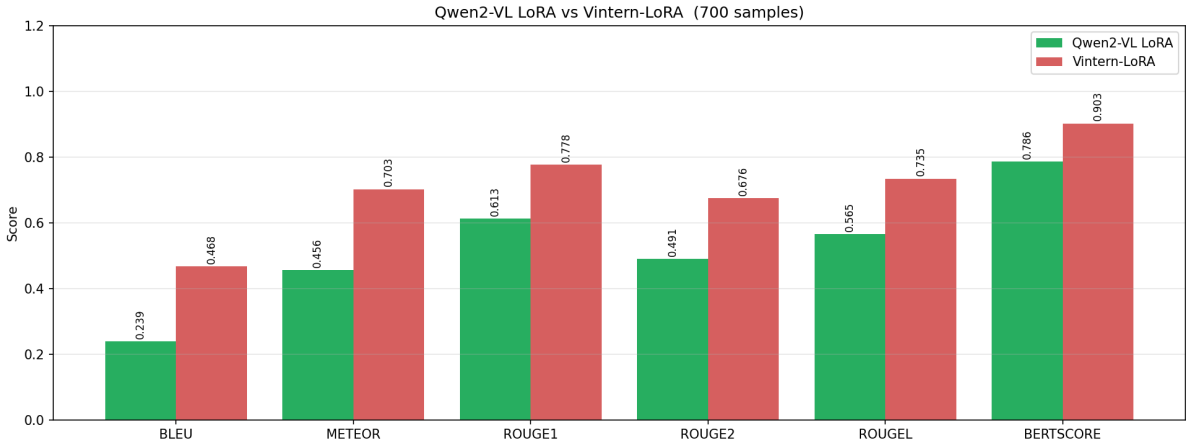


Figure 18: Metric comparison between Qwen2-VL-2B (LoRA) and Vintern-LoRA on the CQA test set (700 samples). Vintern-LoRA consistently achieves higher scores across all metrics despite having approximately half the number of parameters.

Taken together, these results point to a consistent conclusion. When the same fine tuning procedure and training data are applied, Vintern-1B-v2 achieves markedly higher scores than both InternVL2-1B and Qwen2-VL-2B on every metric considered. This outcome strongly suggests that the performance advantage of Vintern-LoRA is not attributable to training methodology or data volume alone, but rather to the model pre training distribution. Vintern-1B-v2 is specifically designed and pre trained for Vietnamese multimodal understanding [4], providing a linguistic and visual prior that is well aligned with the Vietnamese chart question answering task. In contrast, InternVL2-1B and Qwen2-VL-2B, while architecturally capable, are general purpose multilingual models whose pre training is less specialised for Vietnamese, leading to weaker domain adaptation despite receiving

identical fine tuning supervision.

6.1.3 Effect of Structured Table Representation: Paddle+Vintern vs. Vintern Only

Beyond the primary evaluation described above, an additional experiment was conducted to assess whether providing a structured Markdown table representation of the chart extracted via PaddleOCR, as supplementary input to Vintern-1B improves question answering performance compared to passing the chart image alone.

A growing body of research supports the hypothesis that converting chart content into structured textual representations significantly benefits vision language models. Masry et al. [52] demonstrated that models supplied with a corresponding data table alongside the chart image achieve substantially higher accuracy on numerical and relational questions than those relying purely on visual features. Similarly, Liu et al. [72] proposed DePlot, which is a module that translates chart images into linearised data tables and feeds them to a language model, reporting consistent gains across multiple chart QA benchmarks. Han et al. [73] and Ye et al. [74] further showed that grounding the model reasoning in extracted tabular data reduces hallucination and improves factual consistency of the generated answers. These findings collectively motivate the integration of a structured extraction step into the proposed pipeline.

In this work, PaddleOCR [75] is employed to extract the numerical and categorical values from the chart and format them as a Markdown table. This structured representation is then concatenated with the original question prompt before being passed to the fine tuned Vintern-1B model. Figure 19 presents the metric comparison between the two configurations: *Vintern only* (image + question) and *Paddle+Vintern* (image + Markdown table + question).

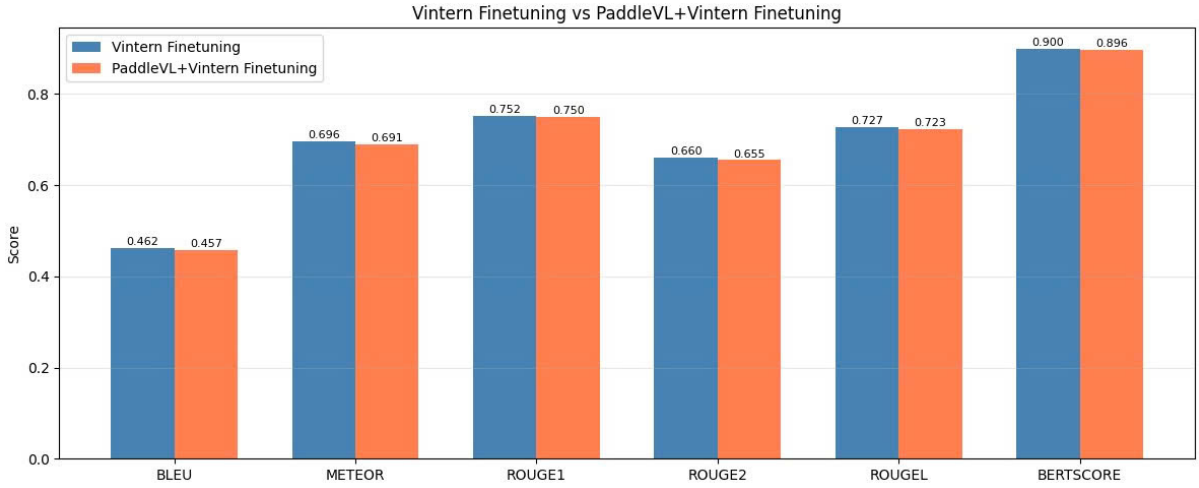


Figure 19: Metric comparison between Vintern-1B with image input only (*Vintern only*) and Vintern-1B augmented with a PaddleOCR extracted Markdown table (*Paddle+Vintern*) across BLEU, METEOR, ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore on the CQA test set.

As shown in Figure 19, the results are mixed. The *Vintern only* configuration achieves higher scores on BLEU (0.477 vs. 0.463), METEOR (0.703 vs. 0.691), ROUGE-2 (0.702 vs. 0.699), ROUGE-L (0.746 vs. 0.744), and BERTScore (0.906 vs. 0.902), while the two configurations perform nearly identically on ROUGE-1 (0.830 vs. 0.831). These results indicate that, contrary to the general trend reported in the literature, the addition of a PaddleOCR extracted Markdown table does not consistently benefit performance in this setting.

Several factors may explain this observation. First, the extraction quality of PaddleOCR is sensitive

to chart formatting, font size, and axis density. Imperfect OCR output including incorrect numerical values, missing labels, or malformed table structure may introduce noise that misleads the model rather than aiding it. Second, since Vintern-1B was fine tuned exclusively on image question answer triplets without any structured table input, the model may not be well calibrated to effectively exploit the additional tabular context during inference. Third, the slight performance gap between the two configurations is small across most metrics, suggesting that the fine tuned model has already learned to extract sufficient information directly from the chart image.

These findings suggest that, for the current pipeline, the image only input is sufficient and that incorporating OCR extracted tables requires additional care, such as fine tuning the model with table augmented samples to realise the potential benefits reported in prior work.

6.2 Discussion

6.2.1 Classification Module

The ResNet-18 classifier achieves a strong overall accuracy of 94.08%, validating the design decision to train the model from scratch on chart specific imagery rather than relying on ImageNet pretrained weights. The high macro F1 score of 0.9407 demonstrates that the model generalises well across all eight chart types, including visually distinct categories such as Heatmap and Pie as well as more ambiguous ones such as Bar and Histogram.

The primary source of classification error lies in the visual similarity between Bar and Histogram charts. Both chart types share rectangular bar structures, and their distinction often relies on subtle contextual cues such as the presence of gaps between bars, axis labelling style, and the nature of the data being represented. These nuances are difficult to capture reliably from image features alone, which accounts for the lower precision observed for the Bar class. Similarly, the low recall for Box charts can be attributed to the relatively abstract and sparse visual structure of box plots, which may be confused with other chart types that also use horizontal or vertical line segments.

From a system perspective, the classification module fulfils its role effectively. It correctly routes the three target chart types, which are Bar, Line, and Pie, with high reliability. Its overall error rate is low enough to avoid introducing significant noise into the downstream question answering pipeline. Since misclassification at this stage would either block valid inputs or pass incorrect chart type information to the reasoning module, the achieved accuracy provides a solid foundation for the end to end system.

6.2.2 Question Answering Module

The results presented in this section address the central research question of whether a compact and domain specific vision language model, fine tuned on a task aligned dataset, can achieve competitive chart question answering performance relative to general purpose multilingual models of comparable or larger scale.

Overall Performance of Vintern-LoRA Across all evaluation metrics, Vintern-LoRA consistently achieves the highest scores among all models evaluated. The fine tuned model demonstrates strong lexical overlap with reference answers as well as high semantic fidelity, confirming that the fine tuning procedure described in Section 4.4.3, when applied to a model whose pre training is well aligned with the target language and domain, yields substantial and consistent gains across all dimensions of text generation quality.

Comparison with Baseline Models Among the five baselines, Qwen2.5-VL-2B achieves the strongest results, reflecting its multilingual visual grounding capacity, while Gemma2-2B performs the weakest

overall, indicating that general purpose language models without task specific adaptation are poorly suited to chart grounded question answering. The performance gap between Vintern-LoRA and even the strongest baseline is consistent and non trivial across all metrics, aligning with findings in related work on domain specific fine tuning of vision language models [76, 77], which demonstrate that task specific supervision on a targeted dataset is more decisive for downstream performance than raw model capacity. The same conclusion is further reinforced by the comparisons with InternVL2-1B and Qwen2-VL-2B discussed in the preceding section, where identical fine tuning conditions were applied yet Vintern-LoRA remained markedly superior.

Limitations Several limitations of the current evaluation should be acknowledged. The metrics employed are reference based measures that may underestimate model quality when the model produces correct but differently phrased responses, which is a known limitation of lexical and embedding based evaluation in open ended generation [78]. Additionally, the test set is drawn from the same distribution as the training data, meaning that generalisation to unseen chart styles or question types remains uncharacterised. The dataset is further constrained to three chart types, leaving performance on less common forms such as heatmaps or scatter plots unevaluated. Finally, the absence of human evaluation limits the assessment of response fluency and factual correctness in practical usage scenarios.

Broader Implications Notwithstanding these limitations, the results demonstrate that it is feasible to build a compact and computationally efficient chart question answering system for Vietnamese by applying parameter efficient fine tuning to a language aligned vision language model, without requiring large scale proprietary data or extensive compute resources. Future work could extend this framework by incorporating a larger and more diverse chart dataset, exploring instruction tuning strategies that improve response fluency, and conducting human evaluation on real world document images.

6.3 Recommendations

Based on the findings from both evaluation components, the following recommendations are proposed for future work and system improvement.

Improving chart classification robustness. The primary confusion between Bar and Histogram charts suggests that incorporating contextual features beyond raw image pixels, such as axis tick patterns or data density, could improve discrimination. One direction is to apply data augmentation strategies specifically targeting the boundaries between visually similar classes, or to introduce a hierarchical classification scheme that first distinguishes broad structural families before resolving fine grained categories.

Expanding the set of supported chart types. The current system supports three chart types, Bar, Line, and Pie, for the end to end QA pipeline. Given that the classifier already recognises eight categories with high accuracy, extending the extraction and reasoning pipeline to cover additional types such as Scatter and Heatmap would substantially broaden the system practical applicability without requiring changes to the classification component.

Improving question answering quality. While fine tuning yields consistent improvements, the use of a larger or more specialised base model, combined with instruction tuning on a more diverse CQA dataset, could further improve performance. Incorporating chain of thought prompting or structured reasoning over the extracted Markdown table may also help the model handle complex numerical questions more reliably.

Improving OCR augmented input quality. Given that the Paddle+Vintern experiment did not yield consistent gains, future work should investigate fine tuning the model with table augmented training samples so that the model learns to leverage structured tabular context alongside the chart image. Additionally, post processing the OCR output to correct common extraction errors before table construction may help reduce noise and better align with the findings reported in the broader chart QA literature [52, 72].

Human evaluation. Future work should complement automatic metric evaluation with human judgement studies to assess answer accuracy, fluency, and relevance from a user perspective. This is particularly important for questions that require interpretation of trends or relative comparisons, where automatic metrics may not adequately reflect answer quality.

Deployment and scalability. For practical deployment beyond the current local workstation setting, containerising the two environment architecture using Docker and deploying on a cloud platform with GPU support would enable the system to serve multiple concurrent users. Implementing a caching layer for repeated chart inputs could further reduce inference latency in high throughput scenarios.

7 Conclusion

7.1 Summary of Findings

The project has successfully developed an automated Chart Question Answering (CQA) system. This system is capable of transforming visual data from pixel formats into searchable and queryable knowledge. The primary objective was to create a lightweight system under 2B parameters with high precision for the Vietnamese language. This goal was achieved through a comprehensive three stage pipeline consisting of classification, structural data extraction, and linguistic inference.

In terms of classification, the ResNet 18 model reached an overall accuracy of 92.43% across eight common chart types. This ensures precise data navigation for all subsequent processing steps. Regarding inference performance, the Vintern 1B v2 model was fine tuned using LoRA techniques. It outperformed popular large language models and multimodal models of similar scale across all evaluation metrics such as BLEU and BERTScore. Furthermore, the integration of Markdown tables from PaddleOCR VL provided a factual data foundation. This integration significantly reduced model hallucination when processing queries that require precise numerical calculations.

7.2 Contributions and Reflections on the Project

The project offers several specific contributions to the field. These include technical advancements through an optimized integration pipeline for resource constrained environments. Additionally, the development of a self constructed Vietnamese chart dataset helps to bridge the language gap in automated chart understanding. From a practical standpoint, the system holds significant potential for corporate report analysis and educational support such as IELTS preparation. It also enhances information accessibility for non expert users.

Reflecting on the execution process, the modular workflow allowed for independent component optimization. It also effectively resolved dependency conflicts between deep learning libraries through the use of isolated environments. Although the GPU memory limit required a reduction in sequence length during training, the project successfully established a strong foundation for interactive visual data analysis using natural language. This constraint partially limited the full exploitation of high resolution dynamic tiling but did not compromise the overall system integrity.

7.3 Limitations and Future Work

Despite these achievements, the system faces certain limitations that provide opportunities for future improvement. Currently, the system is fully optimized for only three primary chart types, which are bar, line, and pie charts. It also maintains a heavy dependence on digital image quality, meaning that low resolution scans remain a significant challenge. Furthermore, current automated evaluation metrics do not fully capture actual fluency and accuracy in complex human centered scenarios.

Future work will focus on expanding capabilities to support complex types such as scatter plots and heatmaps. The team also plans to apply Chain of Thought techniques to enhance arithmetic reasoning. To transition into real world use, the architecture will be containerized via Docker for cloud deployment. This will help to reduce latency and support multiple concurrent users. Finally, human in the loop studies will be conducted to assess the reliability and utility of the system's responses in professional environments. This represents a significant step forward in localizing advanced AI for the Vietnamese context.

References

[3] Zhengzhuo Xu, Sinan Du, Yiyan Qi, Chengjin Xu, Chun Yuan, and Jian Guo. Chartbench: A benchmark for complex visual reasoning in charts, 2024. URL https://arxiv.org/abs/2312.15915.

[1] Aishwarya Agrawal, Jiasen Lu, Stanislaw Antol, Margaret Mitchell, C. Lawrence Zitnick, Dhruv Batra, and Devi Parikh. Vqa: Visual question answering, 2016. URL https://arxiv.org/abs/1505.00468.

[2] Wikipedia contributors. Chart — wikipedia, the free encyclopedia, 2024. URL https://en.wikipedia.org/wiki/Chart. [Online; accessed 23-January-2024].

[4] 5CD-AI. Vintern-1b: An efficient multimodal large language model for vietnamese. *arXiv preprint*, 2024. rlhttps://huggingface.co/5CD-AI/Vintern-1B-v2.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[6] Cheng Cui, Ting Sun, Suyin Liang, et al. Paddleocr-vl: Boosting multilingual document parsing via a 0.9b ultra-compact vision-language model. *arXiv preprint arXiv:2510.14528*, 2025.

[7] Mostafa Dehghani, Basil Mustafa, Josip Djolonga, et al. Patch n' pack: Navit, a vision transformer for any aspect ratio and resolution, 2023.

[8] Baidu ERNIE Team. ERNIE 4.5 technical report. https://ernie.baidu.com/blog/publication/ERNIE_Technical_Report.pdf, 2025.

[9] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. Show and tell: A neural image caption generator. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3156–3164, 2015.

[10] Huaye Qiu, Zi-Yi Dou, Tianlu Wang, Asli Celikyilmaz, and Nanyun Peng. Gender biases in automatic evaluation metrics for image captioning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8358–8375. Association for Computational Linguistics, 2023.

[11] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2425–2433, 2015.

[12] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6700–6709, 2019.

[13] Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1470–1480, 2015.

[14] Ankur Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. Totto: A controlled table-to-text generation dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1173–1186, 2020.

[15] Julian Eisenschlos, Shibi Krichene, and Thomas Muller. Understanding tables with intermediate pre-training. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 281–296, 2020.

[16] Qian Liu, Bei Chen, Jiaqi Guo, et al. Tapex: Table pre-training via learning a neural sql executor. *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=Hc0uO8hEGN.

[17] Zheng Huang, Kai Chen, Jianhua He, Xiang Bai, Dimosthenis Karatzas, Shijian Lu, and CV Jawahar. Icdar2019 competition on scanned receipt ocr and information extraction. In *International Conference on Document Analysis and Recognition (ICDAR)*, pages 1516–1520. IEEE, 2019.

[18] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2200–2209, 2021.

[19] Ryota Tanaka, Kyosuke Nishida, Kosuke Nishida, Taku Hasegawa, Itsumi Saito, and Kuniko Saito. Slidevqa: A dataset for document visual question answering on multiple images. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 13636–13645, 2023.

[20] Cheng Cui, Ting Sun, Suyin Liang, Tingquan Gao, Zelun Zhang, Jiaxuan Liu, Xueqing Wang, Changda Zhou, Hongen Liu, Manhui Lin, Yue Zhang, Yubo Zhang, Handong Zheng, Jing Zhang, Jun Zhang, Yi Liu, Dianhai Yu, and Yanjun Ma. Paddleocr-vl: Boosting multilingual document parsing via a 0.9b ultra-compact vision-language model, 2025. URL https://arxiv.org/abs/2510.14528.

[21] Khang T. Doan, Bao G. Huynh, Dung T. Hoang, Thuc D. Pham, Nhat H. Pham, Quan T. M. Nguyen, Bang Q. Vo, and Suong N. Hoang. Vintern-1b: An efficient multimodal large language model for vietnamese, 2024. URL https://arxiv.org/abs/2408.12480.

[22] Fifth Civil Defender (5CD-AI). Viet chart vqa: A dataset for vietnamese chart question answering. rlhttps://huggingface.co/datasets/5CD-AI/Viet-Chart-VQA, 2024. Accessed: 2024-05-20.

[23] Cornell University. arxiv.org e-print archive. rlhttps://arxiv.org/, 2024. Accessed: April 11, 2026.

[24] Inc. Artifex Software. Pymupdf documentation, 2024. URL https://pymupdf.readthedocs.io/. Python binding for MuPDF, used for PDF page rendering and extraction.

[25] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023. URL https://github.com/ultralytics/ultralytics.

[27] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.

[28] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.

[29] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3320–3328, 2014.

[30] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[31] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48, 2019. doi: 10.1186/s40537-019-0197-0.

[32] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105, 2012.

[33] Justin M. Johnson and Taghi M. Khoshgoftaar. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1):1–54, 2019. doi: 10.1186/s40537-019-0192-5.

[34] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[35] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

[36] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. 2017.

[37] Lutz Prechelt. Early stopping – but when? In Genevieve B. Orr and Klaus-Robert Müller, editors, *Neural Networks: Tricks of the Trade*, pages 55–69. Springer, 1998. doi: 10.1007/3-540-49430-8_3.

[38] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

[39] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.

[40] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL `https://arxiv.org/abs/2106.09685`.

[41] 5CD-AI. Vintern-1b-v2: A compact vietnamese multimodal large language model for ocr, document understanding, and general vqa. rlhttps://huggingface.co/5CD-AI/Vintern-1B-v2, 2024. HuggingFace Model Card.

[42] Zhe Chen, Weiyun Wang, Hao Cao, Yuchen Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, Lixin Zhu, Conghui He, Junjun Shao, Shuai Yan, Yi Wang, Conghui Li, Yu Qiao, Jifeng Dai, and Wenhai Wang. Internvl2: Better than the best—expanding performance boundaries of open-source multimodal models with the best practices for vit, mlp, and llm. *arXiv preprint arXiv:2404.16821*, 2024. URL `https://arxiv.org/abs/2404.16821`.

[43] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2024. URL `https://arxiv.org/abs/2304.08485`.

[44] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *arXiv preprint arXiv:2404.16821*, 2024. URL https://arxiv.org/abs/2404.16821.

[45] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16×16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.11929.

[46] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015. URL https://arxiv.org/abs/1503.02531.

[47] Wenzhe Shi, Jose Caballero, Ferenc Huszar, Johannes Totz, Andrew P. Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. URL https://arxiv.org/abs/1609.05158.

[48] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024. URL https://arxiv.org/abs/2407.10671.

[49] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zelasko, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://arxiv.org/abs/2305.13245.

[50] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL https://arxiv.org/abs/2002.05202.

[51] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. rl-https://github.com/huggingface/peft, 2022. HuggingFace Library.

[52] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279, Dublin, Ireland, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-acl.177.

[53] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[54] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1608.03983.

[55] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. 2016. URL https://arxiv.org/abs/1604.06174.

[56] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Gangadhar Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1710.03740.

[57] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020. URL https://arxiv.org/abs/1910.03771.

[58] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[59] David M. W. Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

[60] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.

[61] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002. doi: 10.3115/1073083.1073135.

[62] Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005.

[63] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pages 74–81, 2004.

[64] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.

[65] Google DeepMind Gemma Team. Gemma 2: Improving open language models at a practical size, 2024. URL https://blog.google/technology/developers/google-gemma-2-.

[66] Meta AI. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models, 2024. URL https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/.

[67] Alibaba Cloud Qwen Team. Qwen2-vl: To see the world more clearly, 2024. URL https://github.com/QwenLM/Qwen2-VL.

[68] Xuan-Phi Nguyen et al. Seallms 3: Open foundation models for southeast asian languages. *arXiv preprint arXiv:2407.19672*, 2024.

[69] Stability AI. Stablelm: Stability ai language models, 2024. URL https://github.com/Stability-AI/StableLM.

[70] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

[71] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[72] Fangyu Liu, Julian Martin Eisenschlos, Francesco Piccinno, Syrine Krichene, Chenxi Pang, Kenton Lee, Mandar Joshi, Wenhu Chen, Nigel Collier, and Yasemin Altun. Deplot: One-shot visual language reasoning by plot-to-table translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8655–8673, Toronto, Canada, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.660.

[73] Yucheng Han, Chi Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Hanwang Zhang. Chartllama: A multimodal llm for chart understanding and generation. *arXiv preprint arXiv:2311.16483*, 2023.

[74] Qinghao Ye, Haiyang Xu, Jiabo Ye, Ming Yan, Haowei Liu, Qi Qian, Ji Zhang, Fei Huang, and Jingren Zhou. mplug-owl2: Revolutionizing multi-modal large language model with modality collaboration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[75] Chenxia Li, Weiwei Liu, Ruoyu Guo, Xiaotian Yin, Kaitao Jiang, Yongkun Du, Yuning Du, Lingfeng Zhu, Baohua Lai, Xiaoguang Liu, Xiangwen Hu, and Dianhai Yu. Pp-ocrv3: More attempts towards an excellent general ocr system. *arXiv preprint arXiv:2206.03001*, 2022.

[76] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[77] Chunyuan Li, Zhe Gan, Zhengyuan Yang, Jianwei Yang, Linjie Li, Lijuan Wang, and Jianfeng Gao. Multimodal foundation models: From specialists to general-purpose assistants. *Foundations and Trends in Computer Graphics and Vision*, 16(1–2):1–214, 2024. doi: 10.1561/0600000101.

[78] Ehud Reiter. A structured review of the validity of bleu. *Computational Linguistics*, 44(3):393–401, 2018. doi: 10.1162/coli_a_00322.